

Administración y configuración

Administración de servicios Web



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: diciembre de 2013

Tabla de contenidos

1	Administración de los servicios Web	7
■	Acerca de este manual	7
	Audiencia	7
■	Arquitectura de los servicios Web	9
	Servicio de presentación Web	9
	Servicio de gestión de sesiones Web	9
	Servicio de seguridad Web	10
■	Introducción a la configuración de los servicios Web	11
	Configuración de los servicios Web	11
	Seguridad	12
	Protección de las páginas de configuración de servicios Web	12
	Archivos temporales del usuario	13
	Configuración de la actualización automática de cachés de tareas y roles	13
■	Configuración de trazas	14
	Ubicación de m4logsystem.cfg	14
	Variables configurables	14
	Trazas generados	15
	Traza de webservices.log	15
■	Páginas de configuración de los servicios Web	16
	Acceso a las páginas de configuración de servicios Web	16
	Parámetros del Servidor de aplicaciones	16
	Parámetros de securización de inicio	17
	Parámetros de seguridad	17
	Parámetros de gestión de ficheros	18
	Parámetros de aplicación	19
	Parámetros de contexto de diagnóstico anidado	19
	Parámetros de la conexión con identificación única del usuario entre aplicaciones de Meta4	20
	Parámetros del servidor Web	20
	Parámetros del producto por defecto	21
■	Activación de filtros	22
	Configuración de filtros	22

Filtro M4BenchFilter	22
Filtro M4AddHeadersFilter	23
Filtro M4WebSSOFilter	24
Parámetros IndirectSSO	25
Filtro M4SOAPAutoGenerationFilter	26
Filtro M4RequestEncodingFilter	27
Filtro M4JrunFilter	28
El filtro M4CompressFilter	28
El filtro M4SecurityHandlerFilter	29
El filtro M4RedirectFilter	30
■ Protección contra CSRF	32
Introducción	32
Configuración	32
Habilitar el "nonce" criptográfico	33
Habilitar el validador del "nonce"	34
Single Sign-On SAML	34
■ Habilitación del Captcha para evitar el bloqueo de cuentas	36
Introducción	36
Configuración	36
Activar parámetro de aplicación de mostrar CAPTCHA y configurar la zona de peligro	37
Verificar la instalación del servlet de KAPTCHA	37
Modificar la página de conexión para integrar el CAPTCHA	38
■ Recuperación de usuario y contraseña	39
Configuración de la recuperación de usuario/contraseña	40
■ Identificación única del usuario entre entornos de Meta4	41
Arquitectura de la identificación única del usuario mediante certificados	41
Configuración de la identificación única Windows/Web	42
Configuración del servidor Web destino	42
Generación de claves pública/privada	43
Configuración de un dominio de uno o varios repositorios con identificación única del usuario	43
Administración de autoridades certificadoras para Internet Explorer cuando el servidor Web destino es seguro	44
Configuración de la identificación única de usuario Web/Web	45
Configuración del servidor Web destino	45
Administración de certificados para servidor Web destino seguro	46
■ Identificación única del usuario desde un portal Web externo	48
Propuesta de identificación única del usuario con un portal externo	48
Esquema de componentes	50
Descripción de las componentes y operaciones	50

Navegador Cliente	50
Contenedor Web	51
Generador de Credenciales	51
Aplicaciones de Meta4: SSE/SSM, PeopleNet Rich Web	51
Generación de la credencial	51
Desarrollos particulares	52
Requerimientos del software	52
Puesta en marcha de la integración de la conexión con identificación única del usuario	52
Generador del repositorio de claves	52
Utilización del API del generador de credenciales	53
Librerías necesarias	53
Invocación del API de generación de credenciales	53
Generación del HTML para la conexión con identificación única del usuario	54
Ejemplos	57
Conexión con identificación única del usuario con aplicación Web	58
Conexión con identificación única del usuario con aplicación Rich Web:	60
■ Configuración de la autorización en servicios Web	64
Introducción a la autorización	64
Configuración del rol por defecto para las aplicaciones Web	64
■ Personalización de recursos por sociedad.	65
Determinar el nodo del árbol de ejecución para la sociedad a personalizar	65
Crear el directorio de personalización (m4custom)	66
Activar el mecanismo.	67
Personalización de páginas JSP	68
Personalización de archivos .properties	69
Personalización de otros archivos (imágenes, javascript, etc.)	70
Personalización de las hojas de estilo (CSS)	71
Personalizar una página JSP previa a la conexión	73
Configuración de la sustitución de literales	74
Sustitución de metadatos	74
Sustitución de datos en páginas	75
Sustitución de datos en ítems de solo lectura	75
■ Pruebas para medir la disponibilidad del servicio.	76
Instalación como aplicación independiente	76
Configuración.	77
Pruebas disponibles.	77
Pruebas de funcionamiento de elementos individuales	78
M4SocketConnectNotifier	78
M4DispatcherConnectNotifier	78
M4XMLLoginNotifier	79

M4SNMPServerNotifier	79
Pruebas de estado general del servicio	80
M4LoginServletNotifier	80
M4GatewayConnectNotifier	80
M4DownloadNotifier	81
Lanzamiento de las pruebas	81

Administración de los servicios Web

Acerca de este manual

En este manual se ofrece la información necesaria para administrar los servicios Web de Meta4.

Para instalar los servicios Web de Meta4, necesita un contenedor Web certificado. Si no desea que el contenedor Web instalado se encargue de las transacciones HTTP, puede utilizar un servidor Web certificado conectado al contenedor Web. Sólo debe utilizar los conectores certificados disponibles para la conexión entre el contenedor Web correspondiente al servidor Web.

Por ejemplo:

- Si utiliza Tomcat como contenedor Web en Windows, Meta4 no certifica ningún servidor Web externo. Debe utilizar Tomcat como servidor Web. Tomcat se encargará de las transacciones HTTP.
- Si utiliza JRun como contenedor Web en Windows o UNIX, puede utilizar un servidor Web externo conectado a JRun, siempre que Meta4 lo incluya en su guía de certificaciones.

Para más información acerca de los conectores certificados y temas relacionados, consulte la *Guía de certificaciones* y la *Guía de instalación de la tecnología base* de Meta4.

Audiencia

Este manual está dirigido a administradores de Meta4 y a personas interesadas en conocer detalles técnicos de la gestión de los servicios Web para el acceso desde clientes ligeros.

Es recomendable haber leído el capítulo La arquitectura cliente/servidor del manual *La Arquitectura de Meta4Objects y sus componentes* y el manual *Guía*

de instalación de la tecnología base para disponer de unos conocimientos básicos que proporcionen una visión global de la aplicación.

También es aconsejable que lea el manual de su servidor Web y su contenedor Web para conocer en profundidad sus posibilidades, y poder compartir información entre dicho servidor y los servicios de Meta4 que se instalan en él.

Asimismo, debe conocer los conceptos básicos utilizados habitualmente en administración de sistemas, redes y bases de datos.

Arquitectura de los servicios Web

En la arquitectura de 3 capas de Meta4, el servidor Web genera las páginas HTML para el cliente a través de los servicios Web de Meta4, que son tres:

- Servicio de presentación Web
- Servicio de gestión de sesiones Web
- Servicio de seguridad Web

Cada servicio se describe a continuación.

Servicio de presentación Web

Este servicio genera páginas HTML de manera dinámica que se visualizan con un navegador Web. La tecnología empleada para generar las páginas se denomina JSP (Java Server Pages). Las JSP utilizan *tags* personalizados de Meta4, extensiones HTML que se ejecutan en el servidor Web y que facilitan el uso de Meta4Objects.

Servicio de gestión de sesiones Web

Este servicio se ocupa de la autenticación y autorización de los usuarios. Cada empleado debe introducir un nombre de usuario y contraseña únicos. Esta información se valida en el Servidor de aplicaciones.

El servicio realiza funciones de balance de carga hacia el Servidor de aplicaciones. Si hay un conjunto de servidores de aplicaciones (o *site dispatchers*) detrás del servidor Web, el servicio de sesión dirige las sesiones de cada usuario a un servidor en concreto. Este proceso ocurre durante la autenticación del usuario.

Además, este servicio hace posible que se mantenga la sesión de usuario. Para ello, transmite una *cookie* entre el cliente HTML y el servidor Web, lo cual identifica al usuario de manera unívoca. El servicio de gestión de sesiones Web también se ocupa de finalizar una sesión de usuario después de un tiempo de inactividad predefinido.

Por la naturaleza de las cookies, si un usuario se conecta varias veces al mismo servidor Web de Meta4 con Internet Explorer 7 u 8 sin haber desconectado la sesiones anteriores, es posible encontrar información de la sesión anterior. Para evitar este comportamiento, si el usuario desea tener varias sesiones diferentes apuntando al mismo servicio Web, es preciso crear una nueva sesión de Internet Explorer y para ello existen varias alternativas:

1. Internet Explorer 8

- Desde la toolbar de IE, seleccionando "File \ New Session". De este modo, ya no se compartirá la sesión con la ventana anterior.
- A través de la ventana de ejecución, mediante la instrucción "Run \ iexplore -nomerge". De este modo, la opción de "New Session" se ejecuta automáticamente y la nueva ventana ya no comparte sesión con las anteriores.

2. Internet Explorer 7

- Introduzca la instrucción "Run \ iexplore".



Como algunas versiones de Internet Explorer comparten sesiones entre varios navegadores, siempre que se trabaje en máquinas de utilización pública se recomienda desconectar la sesión en curso antes de cerrar el navegador, para evitar que otro usuario que se conecte al mismo equipo pueda ver los datos de dicha sesión.

Servicio de seguridad Web

Una vez que la autenticación se realiza correctamente, el servidor Web genera una lista de control de accesos (ACL). El ACL es una lista de los roles disponibles para cada usuario. Cuando un usuario pide una página HTML, el servicio de seguridad comprueba si alguno de sus roles tiene los permisos necesarios para obtener acceso a esa página. Si algún rol los tiene, la página HTML se muestra en el navegador.

El servicio de seguridad también se ocupa de la navegación entre páginas. Es decir, a través de este servicio se localizan las páginas JSP para las que el usuario tiene acceso.

Para más información, consultar el manual "Desarrollo de funcionalidad para el cliente HTML".

Introducción a la configuración de los servicios Web

Después de la instalación, el administrador de sistemas puede necesitar realizar modificaciones en los servicios Web de Meta4.

Por ejemplo:

1. Tareas específicas del contenedor Web, como son:
 - Instalar más *servlets* y JSP en el contenedor Web
 - Administrar u optimizar el contenedor Web (según la documentación del fabricante)
2. Tareas específicas de configuración, como son:
 - Establecer conectividad con servidores de aplicaciones o *dispatchers* nuevos
 - Modificar el comportamiento en tiempo de ejecución (caducidad por inactividad, número máximo de sesiones activas, etcétera)
 - Obtener trazas
 - Configurar funcionalidades adicionales, como la personalización de páginas, etc.

Configuración de los servicios Web

En este apartado se describe cómo modificar aquellos parámetros de los servicios Web específicos de la aplicación de Meta4. Puede hacerlo de dos formas, desde la página de configuración **webappconfig.jsp** o desde el archivo de configuración **configclient.xml**:

1. Abrir la página de configuración (webappconfig.jsp) en un navegador Web, realizar los cambios, guardarlos y reiniciar el contenedor Web. El servidor Web y el contenedor Web tienen que estar en funcionamiento.

EJEMPLO

Español: <http://server.company.com/config/espanol/webappconfig.jsp>.

Inglés: <http://server.company.com/config/english/webappconfig.jsp>.

En la sección siguiente, se indica cómo realizar la autenticación en esta página.

2. Abrir el archivo de configuración (configclient.xml) con un editor de texto, realizar los cambios, guardarlo y reiniciar el contenedor Web.

EJEMPLO

(Windows)

Si durante el proceso de instalación eligió como directorio de instalación la siguiente ruta:

```
C:\Program Files\Meta4\M4WS\default\
```

El archivo con la configuración se encuentra en:

```
C:\Program Files\Meta4\M4WS\default\WEB-INF\classes\properties
```

(UNIX)

Si durante el proceso de instalación eligió como directorio de instalación la siguiente ruta:

```
/opt/meta4/m4webservices/default
```

El archivo con la configuración se encuentra en:

```
/opt/meta4/m4webservices/default/WEB-INF/classes/properties
```

Se recomienda hacer las modificaciones a través de la página de configuración porque evita que se pueda hacer cambios con efectos no deseados en el archivo de configuración.

Si necesita más información sobre algún parámetro específico, consulte el apartado Páginas de configuración de los servicios Web.

Seguridad

Protección de las páginas de configuración de servicios Web

El administrador debe conseguir la seguridad necesaria para evitar el acceso a las páginas de configuración. Puede proporcionar esta protección de dos maneras:

- Externa: utilizando medidas de seguridad válidas para cualquier página en cualquier servidor Web, por ejemplo, hacer que el recurso /config no sea accesible para los usuarios.
- Propia de la aplicación Web: utilizando una contraseña de acceso que se almacena en un fichero cifrado. Esta contraseña es la misma que se utiliza en la inicialización del sitio Web o aplicación Web.
- En un entorno de producción configurado: Eliminando estas páginas.



Para configurar la protección de estas páginas de configuración, consulte el subapartado Parámetros de securización de inicio del apartado Acceso a las páginas de configuración de servicios Web más adelante en este manual.

Archivos temporales del usuario

La aplicación Web genera varios tipos de archivos temporales. Existen archivos temporales bajo el recurso /temp, en la raíz del servidor Web. La convención de nombres para estas carpetas es <usuario>_<xxx>, donde xxx es un número generado aleatoriamente.

Algunos de estos archivos son generados por escritura directa del servidor de aplicaciones. Es preciso asegurarse de que los permisos de escritura en dicho directorio solamente se extienden al usuario que ejecuta el servidor de aplicaciones, mientras que los permisos de lectura solamente se conceden al usuario que ejecuta el contenedor Web.

Es preciso ser especialmente cuidadoso en el caso de que el servidor de aplicaciones y el servidor Web estén instalados en sistemas diferentes.

Configuración de la actualización automática de cachés de tareas y roles

Si desea que la información de tareas y roles que se utilizan en el entorno de cliente ligero (aplicación web para clientes HTML) se refresque automáticamente, debe configurar en el archivo **configclient.xml** el valor de <ServerCheckMultiplier> a un valor entero y distinto de cero.

Este valor es un multiplicador para el parámetro <CacheTimeout> que por defecto vale 60 segundos (un minuto) . De esta forma, si se configura un valor de <ServerCheckMultiplier> a 15, las cachés de los permisos (tareas y roles) se refrescarán cada 15 minutos.

También se realiza con esta frecuencia el refresco de las asignaciones de nodos DNS a sociedades, para entornos con múltiples clientes.

Configuración de trazas

Ubicación de m4logsystem.cfg

La configuración de las trazas de los servicios Web se realiza en el archivo m4logsystem.cfg que se encuentra en el CLASSPATH de las aplicaciones, en:

- Windows: <m4webservices-root>\default\WEB-INF\classes\properties\m4logsystem.cfg
- UNIX: <m4webservices-root>/default/WEB-INF/classes/properties/properties/m4logsystem.cfg

Variables configurables

Editando este archivo se pueden configurar dos variables globales que afectan al comportamiento de las trazas:

- M4_LOG_DIR: directorio en el que se vuelcan los archivos de trazas (en los nombres de los directorios se deben utilizar barras invertidas). Los valores por defecto son:
 - Windows: apunta al directorio c:\temp\m4webservices\logs.
 - UNIX: apunta al directorio /tmp/<usuario>/m4webservices/logs, donde <usuario> es el nombre del usuario.
- M4_TRACE_LEVEL: de menor a mayor severidad, debug | info | warn | error | fatal.

EJEMPLO

El apartado de dichas variables globales aparece en el archivo m4logsystem.cfg de esta manera:

```
#=====
# Global variables.
# - directory names use "/" separator character. Don't use "\".
# - M4_TRACE_LEVEL = debug | info | warn | error | fatal
#=====
M4_LOG_DIR = C:/temp/m4webservices/logs
M4_TRACE_LEVEL = debug
```

Trazas generados

Por defecto se obtienen varios archivos de traza, de los cuales los más importantes son:

- **webservices.log**: contiene las trazas del cliente ligero.
- **connect.log**: contiene un archivo con la comunicación XML entre el servidor Web y el Servidor de aplicaciones.
- **jsp.log**: contiene mensajes obtenidos de las páginas.
- **sessions.log**: registra los usuarios conectados
- **communication.log**: trazas a nivel de transporte



El sistema de trazas está basado en "log4j". Puede consultar la configuración específica (por ejemplo, el comportamiento de "RollingFileAppender", que trabaja junto a la propiedad "MaxFileSize" y con las cuales se pueden limitar el tamaño de los archivos de traza) en la siguiente KB: <http://css.meta4.com/support3x/pwtKbgetsolution.cfm?kid=SP6745>

Traza de webservices.log

Las trazas que aparecen en **webservices.log** suelen tener el siguiente formato:

Fecha y hora [dirección IP, identificador de sesión, fuente de traza - identificador de thread dentro del contenedor web]
Severidad: Texto.

Los parámetros de diagnóstico de contexto anidado ofrecen información adicional sobre el origen de la petición que ha causado la traza.

EJEMPLO

```
Apr 01 13:58:56 [ , , - web-0] DEBUG: LoadTaskList login.  
Apr 01 14:32:07 [172.17.13.143, 11123587167037909082, tctools/  
tc_portal.jsp - web-2] DEBUG: LoadTaskList login.
```

Por defecto algunos de los ficheros (connect.log/communication.log) salen desactivados porque aumentan el tamaño de los ficheros de traza.



Existe más información para configurar el nivel de detalle de esta traza en el subapartado *Parámetros de contexto de diagnóstico anidado* del apartado *Acceso a las páginas de configuración de servicios Web* de este manual.

Páginas de configuración de los servicios Web

Acceso a las páginas de configuración de servicios Web

Todos los parámetros se pueden configurar en el archivo **configclient.xml** o desde las páginas de configuración en la siguiente ruta:

```
http://servidor/config/<idioma>/webappconfig.jsp
```

donde

- **servidor:** es el nombre del servidor que contiene los servicios Web.
- **<idioma>:** es el idioma que se haya seleccionado para la conexión.

Puede parametrizar todas las opciones disponibles, pero recuerde que los cambios que introduzca no tendrán validez hasta que haga clic en el botón **Actualizar**.

Parámetros del Servidor de aplicaciones

Este apartado hace referencia a los parámetros de conexión con cada Servidor de aplicaciones. Cada servidor puede ser un Servidor de aplicaciones o, si tiene instalado el Meta4 Server Site, un *site dispatcher*.

- **Servidor de aplicaciones:** nombre del *host* de la máquina para cada uno de los servidores de aplicaciones o *site dispatcher* (encargados de enviar las solicitudes al Servidor de aplicaciones con menor carga de trabajo) disponibles para conexión desde el servidor Web
- **Puerto del Servidor de aplicaciones:** puerto de conexión normal con el Servidor de aplicaciones elegido
- **Puerto seguro del Servidor de aplicaciones:** puerto seguro del Servidor de aplicaciones elegido
 - **Añadir servidor:** se emplea para agregar servidores de aplicaciones al archivo de configuración. Introduzca los datos del nuevo servidor y haga clic en el botón **Guardar**.

-
- **Eliminar servidor:** se emplea para quitar servidores de aplicaciones del archivo de configuración. Al hacer clic en esta opción, se abre una ventana en la que se muestra el servidor que está marcado para su eliminación; haga clic en el botón **Cerrar**.

Parámetros de securización de inicio

Este apartado hace referencia a los parámetros de securización. Cuando se indica **Securizar sitio**, se pedirá un usuario y una contraseña cifrada para acceder a las páginas también.

- **Usuario de inicialización:** por defecto es WEBADM. El administrador debe saber que el la contraseña del usuario WEBADM está en dos sitios: en el archivo cifrado y en el repositorio.



Importante: Si se cambia en el repositorio la contraseña del usuario WEBADM y esta contraseña no coincide con la que se haya guardado en el archivo cifrado, el entorno Web no se arrancará.

- **Fichero cifrado de inicialización:** la ruta del archivo cifrado de la contraseña del usuario.
- **Securizar sitio:** al hacer clic en **Securizar sitio**, se pedirá un usuario y una contraseña cifrada para acceder a las páginas. El usuario y la contraseña que se debe introducir para entrar a las páginas nuevas, son los mismos que se usan para el arranque, y se guarda en el archivo cifrado.

Parámetros de seguridad

Este apartado hace referencia a los parámetros generales de seguridad de la configuración.

- **SSL del servidor Web al de aplicación:** indicador de conexión segura entre el servidor Web y el Servidor de aplicaciones. Sus posibles valores son:
 - **Yes:** se emplea el protocolo SSL.
 - **No:** no se emplea el protocolo SSL.
- **Certificado SSL:** ruta del certificado necesario para la conexión segura entre el servidor Web y el Servidor de aplicaciones.
- **Usuario invitado:** usuario anónimo de la aplicación. Se utiliza para crear usuarios nuevos mediante un formulario de inscripción.
- **Contraseña del invitado:** contraseña del usuario anónimo de la aplicación.

- **Tiempo de expiración de sesión por inactividad:** número de segundos que una sesión de usuario puede permanecer inactiva, sin peticiones al *servlet*, antes de considerarse concluida.
- **Frecuencia de comprobación de la inactividad:** tiempo que debe transcurrir entre dos comprobaciones diferentes del periodo de inactividad de las sesiones (expresado en segundos).
- **Máximo número de sesiones:** número máximo de sesiones que se pueden establecer con cada servidor Web.
- **Segundos mostrando errores:** en el contexto del login, si se ha realizado una conexión incorrecta con el usuario o con la contraseña, se devuelve una página de error encontrado. El número de segundos indica cuanto tiempo ha de transcurrir antes de volver a la página del login.

Parámetros de gestión de ficheros

Este apartado hace referencia a los parámetros generales de gestión de funcionalidades relacionadas con ficheros.

Los recursos compartidos consideran los componentes del tipo informes, los menús entre otros.

- **Recurso compartido (desde servidor Web):** directorio común del recurso compartido entre el servidor de aplicaciones y el servidor Web vista desde el servidor Web (es decir, cómo ve el servidor Web el directorio compartido).
- **Recurso compartido (desde servidor aplicaciones):** ruta completa del directorio compartido entre el servidor Web y al servidor de aplicaciones, vista desde el servidor de aplicaciones.



Consulte el capítulo *Compartición de ficheros entre el servidor de aplicaciones y el servidor Web* en la *Guía de instalación* para mayor información.

- **Ruta para ficheros en tránsito:** directorio para archivos temporales, utilizado para ver documentos provenientes de campos binarios de la base de datos. Puede configurar este directorio de la siguiente manera:

```
<PATHTEMPFILEMANAGER>c:\temp<\PATHTEMPFILEMANAGER>
```

Este código asigna el directorio temporal del gestor de archivos, será el que se utilizará internamente para los ficheros en tránsito por el servicio de datos.

En el entorno Web si existen unos documentos que necesitan subirse desde un cliente hasta el repositorio mediante el servidor Web, se debe indicar el directorio en el servidor Web donde se guardarán temporalmente estos documentos.

-
- **Carpeta de subida de ficheros:** directorio temporal desde la cual se cargan en el servidor Web.
 - **Máximo tamaño de subida:** tamaño máximo de datos que se pueden cargar desde la base de datos del servidor Web (en caso de peticiones *multipart/form-data*). El valor de este parámetro está expresado en bytes. Para modificar el "máximo tamaño de subida", se puede utilizar la página de configuración (o editar directamente el archivo `configclient.xml`, nodo `<MAXFILESIZE>`) y reiniciar el contenedor al realizar el cambio.

Parámetros de aplicación

Este apartado hace referencia a los parámetros de aplicación de la configuración para una sesión. Consulte el apartado Configuración del servidor Web destino para mayor información acerca de estos parámetros:

- **Secciones generales:** por ejemplo ADMINISTRATION
- **Secciones de usuario:** por ejemplo PORTAL_PARAM, FORMAT, CONFIG_CW

A continuación utilice las botones disponibles para gestionar las secciones: **Añadir sección general**, **Añadir sección de usuario**, **Eliminar sección general** y **Eliminar sección de usuario**.



Consulte el capítulo Parámetros de servicios Web en el manual *HTML Development Framework* para mayor información acerca de estas secciones.

Parámetros de contexto de diagnóstico anidado

Los parámetros de diagnóstico de contexto anidado permiten parametrizar el nivel de detalle que aparece en la traza llamada **webservices.log** acerca del origen de la petición que ha causado la traza.

- **Escribir dirección IP:** si es verdadero, se escribirá la dirección IP desde la que se ha hecho la petición.
- **Escribir fuente traza:** si es verdadero, se escribirá un identificador que apunta a la página desde la que se originó la petición.
- **Escribir ID sesión:** si es verdadero, se escribirá el identificador de sesión que ha hecho la petición

Es decir, que si se ponen a *true* estos parámetros, aparece más información en las trazas sobre quién originó la petición.



Consulte el apartado Configuración de trazas para mayor información acerca de los tipos de trazas que se pueden obtener en el servidor Web.

Parámetros de la conexión con identificación única del usuario entre aplicaciones de Meta4

Este apartado hace referencia a los parámetros que definen la conexión con identificación única del usuario entre dos sistemas Web de Meta4.

- **URL del proveedor de contenido:** ruta a la componente (*servlet*) que proporciona la sesión Web con identificación única del usuario.
- **Almacén de certificados públicos:** ruta del certificado necesario para la conexión segura entre el servidor Web y el servidor de aplicaciones. Consulte el apartado Administración de autoridades certificadoras para Internet Explorer cuando el servidor Web destino es seguro para mayor información acerca de certificados.
- **Contraseña del almacén:** la contraseña de acceso al almacén de certificados.



Para mayor información consulte el apartado Identificación única del usuario entre entornos de Meta4.

Parámetros del servidor Web

Este apartado hace referencia a la configuración del propio servidor Web.

- **Servidor Web:** servidor Web donde se encuentran instalados los servicios Web
- **Puerto del servidor Web:** puerto de conexión del servidor Web; normalmente es el 80
- **Puerto seguro del servidor Web:** puerto seguro de conexión al servidor Web
- **Identificador producto por defecto:** indique el identificador del producto y a utilizar como el defecto.
- **Raíz del cliente ligero:** directorio raíz de las páginas de la aplicación. Se mantiene por compatibilidad hacia atrás, pero no se debe modificar.

-
- **Idioma por defecto:** idioma que se utiliza para redirecciones y mensajes de error si no se especifica uno como parámetro del formulario de inicio de sesión. También se utilizará este idioma en el Meta4Object de sesión en el Servidor de aplicaciones.
 - **Parámetros del producto por defecto:** para proceder a definir estos parámetros, haga clic en **Abrir** para proceder a completar los Parámetros del producto por defecto.
 -

Parámetros del producto por defecto

En este apartado se configura todos los parámetros del producto por defecto que incluyen el tratamiento por idiomas, las páginas a utilizar por defecto:

- **Página de portal por defecto:** indique la página del portal a utilizar por defecto.
- **Hoja de estilo por defecto:** indique la hoja de estilo a utilizar por defecto.
- **Idiomas:** para cada idioma se define una serie de parámetros que se describen en el apartado Configuración de carpetas por idiomas.

Configuración de carpetas por idiomas

Este apartado hace referencia a la información que se va a emplear con los usuarios en función del idioma elegido. Al final de la lista de idiomas disponibles, existe un apartado de idioma genérico que puede configurar si no existe el idioma que desea.

Cada idioma consta de los siguientes parámetros de configuración:

- **Carpeta de idioma:** directorio identificativo del idioma.
- **Configuración regional:** configuración regional utilizada cuando se emplea el idioma que se está definiendo. Los valores posibles son los de las configuraciones regionales de Java: *en*, *es*, *fr*, etcétera. Esta configuración regional se utiliza para los mensajes de error.
- **Página de conexión:** página de conexión, a la que se redirigirá tras el periodo de inactividad.
- **Página de errores:** página que se muestra en caso de error.
- **Cambio de contraseña:** página intermedia a la que se redirige cuando la contraseña del usuario está próxima a expirar.

Activación de filtros

En la aplicación Web de Meta4 para clientes HTML, a los servicios Web proporcionados por servlets o páginas JSP se adjunta cierto número de filtros que realizan tareas complementarias.

Entre otras tareas, estos filtros se encargan de:

- Establecer cabeceras HTTP: M4JRunFilter (sólo JRun), M4AddHeadersFilter, M4RequestEncoding, etc.
- Permitir la medición de tiempos: M4BenchFilter
- Asegurar la compresión de datos: M4CompressFilter (sólo JRun)
- Funcionalidades complementarias: M4SoapAutogenerationFilter, M4RedirectFilter, M4WebSSOFilter, etc.

Configuración de filtros

Los filtros se configuran en el fichero **web.xml**. Algunos de ellos están por defecto activados (enable=true) aunque otros es preciso activarlos (false -> true).

El orden correcto de definición y mapeo de dichos filtros en el archivo **web.xml** es el siguiente:

- M4BenchFilter
- M4AddHeadersFilter
- M4WebSSOFilter
- M4SOAPAutoGenerationFilter
- M4RequestEncodingFilter
- M4JrunFilter
- M4CompressFilter
- M4SecurityHandlerFilter
- M4RedirectFilter

Filtro M4BenchFilter

El **filtro de medida del rendimiento** se utiliza para medir la duración de las transacciones realizadas por la aplicación web para clientes HTML.

A continuación se muestra un ejemplo del código de definición y mapeo del

filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 benchmarking filter (should be first) -->
<filter>
<filter-name>M4BenchFilter</filter-name>
<filter-class>com.meta4.filters.bench.M4BenchFilter</filter-class>
</filter>
<!-- [END] Meta4 benchmarking filter -->

<!-- [BEGIN] Meta4 benchmarking filter -->
<filter-mapping>
<filter-name>M4BenchFilter</filter-name>
<url-pattern>/*</url-pattern>
<dispatcher>REQUEST</dispatcher>
</filter-mapping>
<!-- [END] Meta4 benchmarking filter -->
```

Filtro M4AddHeadersFilter

El **filtro de establecimiento de cabeceras** se utiliza para establecer ciertas cabeceras HTTP de forma global para toda la aplicación web para clientes HTML. Su finalidad es evitar que existan páginas dentro de una aplicación web que carezcan de las directivas necesarias para evitar su almacenamiento en cachés o infraestructuras de proxies.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 AddHeader filter - before redirect, compress, etc
filters -->
<filter>
<filter-name>M4AddHeadersFilter</filter-name>
<filter-class>com.meta4.filters.addheaders.M4AddHeadersFilter</
filter-class>
<init-param>
  <param-name>enable</param-name>
  <param-value>>false</param-value>
```

```
</init-param>
</filter>
<!-- [END] Meta4 AddHeader filter -->

<!-- [BEGIN] Meta4 AddHeaders filter before redirect, compress, etc
filters -->
<filter-mapping>
<filter-name>M4AddHeadersFilter</filter-name>
<url-pattern>*.jsp</url-pattern>
</filter-mapping>
<!-- [END] Meta4 AddHeaders filter -->
```

Una vez activado el filtro, se añadirán de forma automática las siguientes cabeceras:

```
this.setHeader("Pragma", "no-cache");           //HTTP 1.0
this.setHeader("Cache-Control", "no-cache");     //HTTP 1.1
this.setHeader("Cache-Control", "no-store");     //HTTP 1.1
this.setDateHeader("Expires", -1);
```

Filtro M4WebSSOFilter

El **filtro de Single Sign-on** (Identificación Única del Usuario) se utiliza para permitir el SSO SAML. Para ello debe activarse, ya que está desactivado por defecto.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 Single Sign On filter -->
<filter>
<filter-name>M4WebSSOFilter</filter-name>
<filter-class>com.meta4.filters.websso.M4WebSSOFilter</filter-class>
<init-param>
<param-name>netpattern</param-name>
<param-value>^/hrprof</param-value>
</init-param>
<init-param>
<param-name>adbpattern</param-name>
<param-value>^/adb</param-value>
```

```

</init-param>
<init-param>
<param-name>richwebpattern</param-name>
<param-value>^/rw</param-value>
</init-param>
<init-param>
<param-name>esspattern</param-name>
<param-value>/</param-value>
</init-param>
<init-param>
<param-name>enable</param-name>
<param-value>true</param-value>
</init-param>
</filter>
<!-- [END] Meta4 Single Sign On filter -->

<!-- [BEGIN] Meta4 shiboleth filter -->
<filter-mapping>
<filter-name>M4WebSSOFilter</filter-name>
<url-pattern>*.jsp</url-pattern>
</filter-mapping>
<!-- [END] Meta4 shiboleth filter -->

```

Parámetros IndirectSSO

Además de activar el filtro de Single Sign-on en los entornos que lo necesiten utilizar, es necesario configurar los parámetros de conexión unificada IndirectSSO en el archivo **configclient.xml**. Así, por ejemplo, si se utiliza Single Sign-on SAML, para el correcto funcionamiento del Self-Service del Empleado (SSE) es necesario especificar en dicho archivo un patrón que refleje correctamente cuál es la página de conexión del portal (aquella que contiene el servlet de login) a través del parámetro portalloginpattern.

A continuación se muestra un ejemplo de configuración:

EJEMPLO

```

<IndirectSSO>
<filepath>ca-saml.p12</filepath>
<keypass>changeit</keypass>
<interval>600</interval>

```

```
<margin>60</margin>
<portalprotocol>https</portalprotocol>
<portalport>443</portalport>
<portalloginpattern>sse_generico/generico_login.jsp</portalloginpattern>
<portaldefaulturi>/servlet/CheckSecurity/JSP/sse_generico/ssco_portal.jsp</portaldefaulturi>
<updatesurl>https://www.meta4globalhr.com/hr</updatesurl>
<richappserver>localhost</richappserver>
<net>
<netupdatesurl>https://meta4.meta4globalhr.com/hrprof</netupdatesurl>
<netupdservice>https://meta4.meta4globalhr.com/hrprof</netupdservice>
<netappserver>DefaultServer</netappserver>
</net>
</IndirectSSO>
```

Filtro M4SOAPAutoGenerationFilter

El **filtro de SOAP** se utiliza para dos propósitos: realizar la generación automática de los servicios SOAP que se construyen a partir de Meta4Objects que exportan métodos de negocio, y corregir el WSDL que emite Axis 1.x para que sea compatible con otras plataformas de consumo de servicios SOAP.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 soapservices filter -->
<filter>
<filter-name>M4SOAPAutoGenerationFilter</filter-name>
<filter-
class>com.meta4.filters.soapservices.M4SOAPAutoGenerationFilter</
filter-class>
<init-param>
<param-name>enable</param-name>
<param-value>true</param-value>
</init-param>
</filter>
```

```
<!-- [END] Meta4 soapservices filter -->

<!-- [BEGIN] Meta4 soapservices filter -->
<filter-mapping>
<filter-name>M4SOAPAutoGenerationFilter</filter-name>
<url-pattern>/services/*</url-pattern>
</filter-mapping>
<filter-mapping>
<filter-name>M4SOAPAutoGenerationFilter</filter-name>
<url-pattern>/servlet/AxisServlet</url-pattern>
</filter-mapping>
<!-- [END] Meta4 soapservices filter -->
```

Filtro M4RequestEncodingFilter

El **filtro de internacionalización** se utiliza para convertir los datos entrantes al formato en que el contenedor web los recoge. Como el contenedor web suele trabajar en ISO-8859-1, cuando se trabaja en entornos Unicode, se encarga de transformar los datos de entrada de UTF-8 a ISO-8859-1.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 i18n filter -->
<filter>
<filter-name>M4RequestEncodingFilter</filter-name>
<filter-class>com.meta4.filters.i18n.M4RequestEncodingFilter</filter-
class>
<init-param>
<param-name>enable</param-name>
<param-value>true</param-value>
</init-param>
</filter>
<!-- [END] Meta4 i18n filter -->

<!-- [BEGIN] Meta4 i18n filter -->
<filter-mapping>
<filter-name>M4RequestEncodingFilter</filter-name>
```

```
<url-pattern>*.jsp</url-pattern>
</filter-mapping>
<!-- [END] Meta4 i18n filter -->
```

Filtro M4JrunFilter

El **filtro para JRun**, solo definido en este contenedor, corrige un comportamiento en el servicio de ficheros que ofrece este contenedor web, por el cual no se ofrecían las fechas de las páginas JSP, de forma que siempre se ejecutaban aunque el desarrollador deseara que su contenido fuera cacheado en el navegador.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 control filter for JRun 4.0 -->
<filter>
<filter-name>M4JRunFilter</filter-name>
<filter-class>com.meta4.filters.M4JRunFilter</filter-class>
</filter>
<!-- [END] Meta4 control filter for JRun 4.0 -->

<!-- [BEGIN] Meta4 control filter for JRun 4.0 -->
<filter-mapping>
<filter-name>M4JRunFilter</filter-name>
<url-pattern>/*</url-pattern>
</filter-mapping>
<!-- [END] Meta4 control filter for JRun 4.0 -->
```

El filtro M4CompressFilter

El **filtro de compresión**, solo definido en JRun, comprime la salida de las páginas JSP para mejorar el rendimiento de las peticiones HTTP. Este filtro solo se ofrece para JRun porque otros contenedores web, como Tomcat, proporcionan esta funcionalidad ellos mismos.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```
<!-- [BEGIN] Meta4 compression filter configuration -->
<filter>
<filter-name>M4CompressFilter</filter-name>
<filter-class>com.meta4.filters.M4CompressFilter</filter-class>
<init-param>
<param-name>bufferSize</param-name>
<param-value>256</param-value>
</init-param>
<init-param>
<param-name>enable</param-name>
<param-value>true</param-value>
</init-param>
<init-param>
<param-name>output</param-name>
<param-value>true</param-value>
</init-param>
</filter>
<!-- [END] Meta4 compression filter configuration -->

<!-- [BEGIN] Meta4 compression filter configuration -->
<filter-mapping>
<filter-name>M4CompressFilter</filter-name>
<url-pattern>*.jsp</url-pattern>
</filter-mapping>
<!-- [END] Meta4 compression filter configuration -->
```

El filtro M4SecurityHandlerFilter

El **filtro de manejo de la seguridad** se encarga de diversas operaciones relacionadas con la seguridad, como protección contra CSRF (puede ver más información al respecto en el apartado Protección contra CSRF de este manual).

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```

<!-- [BEGIN] Meta4 security filter (must be after compression) -->
<filter>
<filter-name>M4SecurityHandlerFilter</filter-name>
<filter-
class>com.meta4.filters.security.jrun.M4SecurityHandlerFilter</
filter-class>
<init-param>
<param-name>enable</param-name>
<param-value>>false</param-value>
</init-param>
</filter>
<!-- [END] Meta4 security filter -->

<!-- [BEGIN] Meta4 security filter -->
<filter-mapping>
<filter-name>M4SecurityHandlerFilter</filter-name>
<url-pattern>*.jsp</url-pattern>
</filter-mapping>
<filter-mapping>
<filter-name>M4SecurityHandlerFilter</filter-name>
<url-pattern>*.js</url-pattern>
</filter-mapping>
<!-- [END] Meta4 security filter -->

```

El filtro M4RedirectFilter

El ***filtro de personalización*** se utiliza para realizar la personalización y la sustitución de literales, controlando asimismo ciertas opciones relacionadas con el contenido y las fechas de los ficheros personalizables y/o traducibles.

A continuación se muestra un ejemplo del código de definición y mapeo del filtro (primero la definición y después el mapeo) en el archivo **web.xml**:

EJEMPLO

```

<!-- [BEGIN] Meta4 customization filter configuration -->
<filter>
<filter-name>M4RedirectFilter</filter-name>

```

```
<filter-class>com.meta4.filters.redirect.M4RedirectFilter</filter-
class>
<init-param>
<param-name>enable</param-name>
<param-value>>false</param-value>
</init-param>
<init-param>
<param-name>exclude</param-name>
<param-value>^/temp/</param-value>
</init-param>
<init-param>
<param-name>expire1</param-name>
<param-value>\.png$;86400</param-value>
</init-param>
<init-param>
<param-name>expire2</param-name>
<param-value>\.gif$;86400</param-value>
</init-param>
<init-param>
<param-name>expire3</param-name>
<param-value>\.jpg$;86400</param-value>
</init-param>
<init-param>
<param-name>expire4</param-name>
<param-value>\.css$;3600</param-value>
</init-param>
<init-param>
<param-name>expire5</param-name>
<param-value>\.js$;3600</param-value>
</init-param>
</filter>
<!-- [END] Meta4 customization filter configuration -->

<!-- [BEGIN] Meta4 customization filter configuration -->
<filter-mapping>
<filter-name>M4RedirectFilter</filter-name>
<url-pattern>*</url-pattern>
</filter-mapping>
<!-- [END] Meta4 customization filter configuration -->
```

Protección contra CSRF

Introducción

La vulnerabilidad web conocida como CSRF (Cross Site Request Forgery, o Falsificación de Peticiones Entre Sitios) se produce cuando una aplicación web confía en las peticiones sin tener precauciones sobre el origen de las mismas.

Esto, en general, ocurre en la mayoría de las aplicaciones en las que el usuario está autenticado mediante una cookie almacenada en el navegador del usuario, y no se tiene ninguna precaución especial adicional.

Si el usuario accede a un sitio malicioso que, de forma inadvertida, es capaz de generar una petición HTTP especial y consigue que el usuario la dispare mientras está autenticado en la aplicación original, podría generar una acción no deseada.

Para solucionar este problema se puede proceder de dos formas:

1. Manualmente: uno de los campos podríamos encriptarlo y desencriptarlo en la página destino (por ejemplo, uno de los campos obligatorios).
2. Con una utilidad automática: existe la posibilidad de habilitar un mecanismo de protección contra CSRF, que consta de dos piezas que funcionan de forma complementaria. La primera es un generador de claves, que crea claves criptográficas únicas por sesión (nonce, de “number used once”) para todas las páginas de aplicación y archivos JavaScript, y la segunda es un validador de las mismas. En el caso de que se solicite un recurso sin la clave que lo protege, el sistema puede configurarse para mostrar un error o para desconectar al usuario de la sesión actual.

Configuración

Por defecto esta funcionalidad está desactivada. Si se activa, hay que poner en funcionamiento también el filtro de personalización para que se encargue de que los ficheros JavaScript se envíen al navegador del cliente con cabeceras HTTP que no afecten al rendimiento de la aplicación.

Para configurar el mecanismo de protección, siga este procedimiento:

1. Habilite el “nonce” criptográfico.
2. Habilite el validador del “nonce”.
3. Configure la ruta a la página de login para Single Sign-On SAML.

Habilitar el "nonce" criptográfico

Para habilitar el generador del "nonce" criptográfico, deberá asegurarse de que la siguiente propiedad esté activada:

(WEB-INF/web.xml > M4SecurityHandlerFilter > enable = true)



Para el correcto funcionamiento del filtro M4SecurityHandlerFilter es necesario que el filtro M4RedirectFilter (personalización de archivos) esté activado.

A continuación se muestra un ejemplo de cómo habilitar el "nonce" criptográfico en JRun:

EJEMPLO DE HABILITACIÓN EN JRUN

```
<!-- [BEGIN] Meta4 Security Handler Filter -->
<filter>
    <filter-name>M4SecurityHandlerFilter</filter-name>
    <filter-
class>com.meta4.filters.security.jrun.M4SecurityHandlerFilter
    </filter-class>
    <init-param>
        <param-name>enable</param-name>
        <param-value>true</param-value>
    </init-param>
</filter>
```

A continuación se muestra un ejemplo de cómo habilitar el "nonce" criptográfico en Tomcat:

EJEMPLO DE HABILITACIÓN EN TOMCAT

```
<!-- [BEGIN] Meta4 Security Handler Filter -->
<filter>
    <filter-name>M4SecurityHandlerFilter</filter-name>
    <filter-
class>com.meta4.filters.security.M4SecurityHandlerFilter
    </filter-class>
    <init-param>
        <param-name>enable</param-name>
```

```

        <param-value>true</param-value>
    </init-param>
</filter>

```

Habilitar el validador del "nonce"

Para habilitar el validador del "nonce", la propiedad active debe estar a true en m4csrfvalidator.properties:

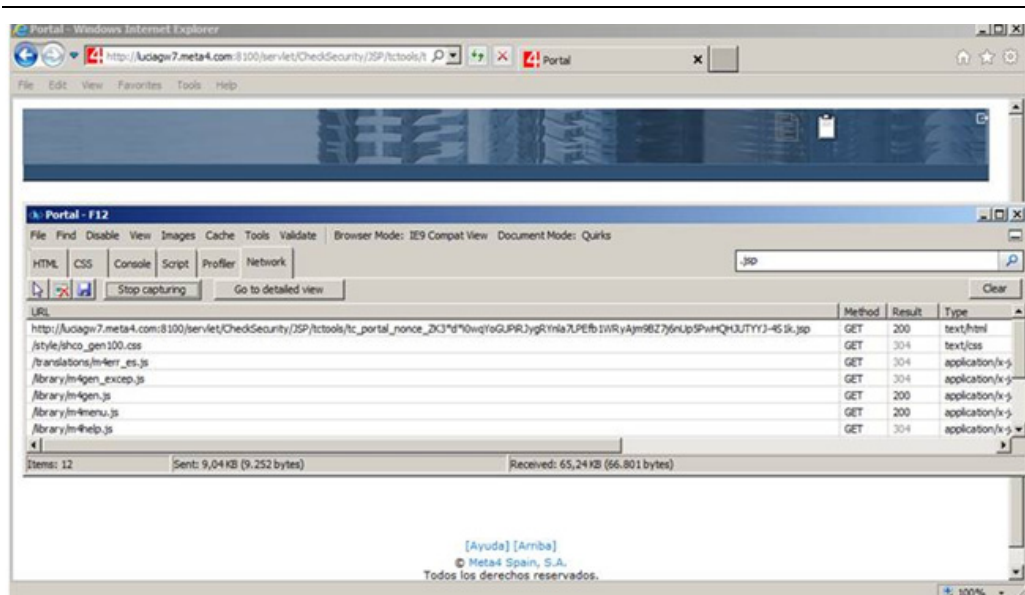
(WEB-INF/classes/properties/m4csrfvalidator.properties, active = true)

La propiedad dologoutonerror sirve para configurar el comportamiento del filtro cuando entra una página sin token criptográfico. Si está activo, entonces el sistema deslogará al usuario que lo haya intentado. En este deslogado se basa el comportamiento correcto de ciertos entornos de Single Sign-On.

Single Sign-On SAML

En el Single Sign-On SAML, para el correcto funcionamiento del ESS es necesario especificar la ruta exacta a la página de login en el archivo configclient.xml.

Figura 1. Control de activación de la protección contra CSRF





El filtro `M4SecurityHandlerFilter` modifica al vuelo todos los archivos JavaScript, añadiendo en los enlaces a páginas JSP tokens criptográficos. Por lo tanto, si se encontraran copias cacheadas de dichos archivos en el navegador del cliente o en proxies intermedios, se podrían dar problemas iniciales (al comenzar a utilizar la protección contra CSRF). Es necesario borrar las cachés del navegador y verificar la caducidad de recursos en el proxy para evitar este tipo de interferencia. El propio filtro `M4SecurityHandlerFilter` se encarga de la correcta expiración de los archivos JavaScript una vez activado.

Habilitación del Captcha para evitar el bloqueo de cuentas

Introducción

Existe la posibilidad de habilitar un CAPTCHA para la página de conexión de las aplicaciones web para clientes HTML. En dichas aplicaciones, si se habilita esta funcionalidad mediante un parámetro de aplicación global, a aquellos usuarios que excedan un cierto número de intentos, también configurable, se les solicitará que, además del usuario y la contraseña, introduzcan unos caracteres que aparecerán como una imagen en la pantalla. Esto intentará evitar, en la medida de lo posible, que de forma automática, un robot pueda utilizarse para bloquear usuarios con facilidad tras intentar una contraseña errónea un número grande de veces.

Para implementar un sistema de seguridad que frene la automatización del bloqueo de usuarios, el servlet de login puede protegerse mediante la implementación de CAPTCHA llamada KAPTCHA, que es un servlet que produce una imagen que el usuario debe introducir además de su usuario y contraseña.

Si se implementa esta protección, el servlet, antes de que el usuario se conecte, mira si el usuario está en zona de peligro. Un usuario está en zona de peligro si el número de intentos que lleva consumidos (valor del campo WTRY_CONNECT de la tabla M4RSC_APPUSER) es mayor que un cierto parámetro de aplicación que indica cuántos intentos se tienen antes de mostrar la imagen.

Si se desea máxima seguridad, se puede configurar este valor a 1, y siempre que un usuario se conecte erróneamente, tanto si es un usuario inexistente, como un usuario bloqueado, como un usuario existente pero aún no bloqueado, se le mostrará la imagen del desafío/respuesta.

Configuración

Para habilitar la anti-automatización en la página de conexión de las aplicaciones para cliente HTML es necesario llevar a cabo los pasos que se detallan a continuación.

Activar parámetro de aplicación de mostrar CAPTCHA y configurar la zona de peligro

El CAPTCHA se habilita mediante un parámetro de aplicación de ámbito global (THINCLIENT.ADMINISTRATION.DISPLAY_CAPTCHA = 1) y el número de intentos que se dejarán al usuario antes de mostrar el mismo, se configura en un parámetro de aplicación que puede configurarse en el ámbito que se desee, sea global o sea local (SECURITY.USER_LOCKING.NUM_ATTEMPTS_DANGER). Se recomienda que el valor sea 1 para máxima seguridad.

Verificar la instalación del servlet de KAPTCHA

En WEB-INF\web.xml de la aplicación web para cliente HTML deben constar tanto el servlet de KAPTCHA como su mapeo.

Las propiedades del servlet de KAPTCHA se manejan en este archivo, según la documentación que aparece en la URL <http://code.google.com/p/kaptcha/wiki/ConfigParameters>.

EJEMPLO

```
<servlet>
    <servlet-name>Kaptcha</servlet-name>
    <servlet-class>com.google.code.kaptcha.servlet.KaptchaServlet
</servlet-class>
    <init-param>
        <param-name>kaptcha.border</param-name>
        <param-value>no</param-value>
    </init-param>
    <init-param>
        <param-name>kaptcha.textproducer.font.color</param-name>
        <param-value>black</param-value>
    </init-param>
    <init-param>
        <param-name>kaptcha.textproducer.char.space</param-name>
        <param-value>5</param-value>
    </init-param>
</servlet>

<servlet-mapping>
    <servlet-name>Kaptcha</servlet-name>
```

```
<url-pattern>/images/kaptcha.jpg</url-pattern>  
</servlet-mapping>
```

Modificar la página de conexión para integrar el CAPTCHA

El servlet de login, en caso de que el usuario detecte que está en zona de peligro, impedirá logarse al usuario y redirigirá a la página de login con un atributo establecido en la sesión web, SecurityAutomationControl.PARAM_REQUIRE_CAPTCHA con valor True.

Si se ha equivocado en el CAPTCHA, redirigirá a la página de login con un atributo llamado SecurityAutomationControl.PARAM_RETYPE_CAPTCHA con valor True.

Para que no tenga que teclear de nuevo el usuario, sólo la contraseña, en la sesión HTTP se queda el último usuario que resultó estar en peligro de bloquearse en el último intento, SecurityAutomationControl.ATT_LOCKING_DANGER.

Este valor se establece en el servlet de login y solo se borra cuando un usuario ha tenido éxito en su conexión.

Recuperación de usuario y contraseña

Si un empleado no recuerda el nombre de usuario o la contraseña de acceso a las aplicaciones Web para clientes HTML de Meta4 (como, por ejemplo, SSE/SSM), en la página de acceso existe un enlace para recuperar el nombre de usuario y la contraseña.

Dicho enlace, llamado "He olvidado mi usuario/contraseña", permite recuperar el nombre de usuario y obtener una nueva contraseña. Al hacer clic en él, se solicitan una serie de datos identificativos, que pueden configurarse según sea conveniente, para identificar al usuario y poder realizar el proceso de recuperación.

Es imprescindible que el administrador de las aplicaciones Web configure cuáles son los campos específicos que en su organización constituyen los identificadores del usuario. Estos campos deben ser lo más precisos posibles para que la búsqueda arroje como resultado un único usuario. Por ejemplo, en una empresa española podría indicarse el número de identificación fiscal o pasaporte, además del nombre, apellidos, fecha de nacimiento, etc.



Tenga en cuenta que los identificadores de los campos deben coincidir con identificadores de campos de la tabla de personas (STD_PERSON).

A través de dichos datos identificativos, se busca a la persona y a partir de ella a los usuarios de aplicación que tiene asociados. La persona tiene que tener un periodo dado de alta.

Una vez generado el proceso, se crea un correo electrónico con la información necesaria para poder llevar a cabo el cambio de contraseña. Este correo va dirigido al usuario (si tiene correo) o a su responsable (por ejemplo, en el departamento de RR.HH.) si no lo tiene. Si el correo se envía a RR.HH., el proceso asociado a la generación de la nueva contraseña es automático; es decir, el sistema genera un fichero con la información correspondiente. A su vez, si el correo va dirigido al usuario, el proceso asociado a la generación de la nueva contraseña es manual, y el usuario debe introducir la nueva contraseña para realizar el cambio.

Si la persona sólo ha olvidado su usuario, no tiene que realizar ningún tipo de acción porque en el correo figurará su usuario/os con el/los que podrá conectarse. Por el contrario, si ha olvidado la contraseña, deberá cambiar o resetear la contraseña mediante el/los enlace/s enviados (en cada enlace se indicará el usuario/sociedad/rol por defecto).

Configuración de la recuperación de usuario/contraseña

Para llevar a cabo el proceso de recuperación de usuario y/o contraseña, se ha creado el Meta4Object SRTC_FORGET_PWD, así como una serie de parámetros de aplicación que se configuran desde PeopleNet/Configuración de aplicación/Parámetros de aplicación/Parámetros avanzados.

Una vez en la ventana de Parámetros avanzados, acceda a la aplicación "THINCLIENT" (Parámetros de cliente ligero) y a la sección "FORGET_PWD" (Olvidar contraseña). Dentro de esa sección, existen los siguientes parámetros:

- **FIELDS:** campos identificativos que van a ser solicitados a un usuario para poder localizar la persona asociada a él. Los diferentes campos deben separarse por comas ",".
- **FROM:** cuenta de correo electrónico que aparece en el remitente del correo.
- **MAIL_RRHH:** cuenta de correo del responsable de recursos humanos que debe aceptar el cambio de contraseña de un usuario.

Identificación única del usuario entre entornos de Meta4

En esta sección se describe la configuración del procedimiento de identificación única del usuario (Single Sign-On, SSO) entre entornos de Meta4 mediante el empleo de certificados.

La identificación única del usuario es un esquema que permite que una autenticación previa realizada siguiendo un procedimiento cualquiera no tenga que repetirse para dar acceso a otros entornos informáticos. En la práctica implica que el usuario no tenga que volver a introducir, por ejemplo, su usuario y contraseña para tener acceso a distintos sistemas.

El procedimiento descrito en esta sección sirve para establecer identificación única de un usuario que ya se encuentra conectado en un sistema de Meta4 y desea conectarse a otro sistema de Meta4 diferente.

Cuando la identificación única del usuario debe realizarse con un sistema externo, puede consultar la información sobre identificación única del usuario desde un portal externo, que se describe en el próximo apartado de este manual, así como la opción de configurar la identificación única del usuario desde un portal externo mediante el empleo de dominios de autenticación.

Puede consultar dicha configuración en el manual *Single_Sign_On_Web.pdf*.

Arquitectura de la identificación única del usuario mediante certificados

El servidor de aplicaciones de Meta4 es capaz de emitir certificados firmados con una clave privada que se encuentra en el repositorio de base de datos.

Por lo tanto, si un usuario es capaz de obtener una sesión en la aplicación, puede utilizarla para obtener un certificado que le resulte válido en otro servidor de aplicaciones.

Cuando el cliente ya autenticado envía a un servidor de aplicaciones diferente el certificado, el servidor de aplicaciones destino comprueba que el certificado ha sido firmado por un servidor de aplicaciones en quien se confía (es decir, ha construido el certificado a partir de la misma clave privada) y concede una sesión al usuario sin que este tenga que introducir de nuevo una contraseña.

Esto es posible tanto si el cliente es Windows (identificación única del usuario de cliente Windows a servidor Web) como si el cliente es cliente ligero o HTML (identificación única del usuario de servidor Web a servidor Web).

Las únicas restricciones para implantar la identificación única del usuario siguiendo este esquema de certificados entre aplicaciones de Meta4 cuando los repositorios de base de datos son diferentes, por tanto, son:

- Que los mismos nombres de usuario existan en los dos repositorios.
- Que los dos repositorios sean firmados con la misma clave privada.

Configuración de la identificación única Windows/Web

Para hacer posible la identificación única Windows/Web, la primera vez que se inician los servicios Web de Meta4 se realiza una configuración automática por defecto de los parámetros necesarios.

Si fuera necesario modificar la configuración por defecto, se indican a continuación las tareas que pueden ser necesarias para realizar el cambio.

Configuración del servidor Web destino

El cliente Windows necesita saber el nombre y puerto del servidor Web destino de la identificación única del usuario. Esta información se extrae de parámetros de la aplicación, que se guardan en base de datos.

Para facilitar la primera instalación del sitio Web, el primer servidor Web que se conecte con éxito a un repositorio de base de datos que no tenga configurado el nombre del servidor Web destino, se inscribirá como servidor destino por defecto.

Si desea cambiar los parámetros de aplicación relativos al servidor Web destino, bien porque se modifica el nombre del servidor, el puerto, o el protocolo, o bien porque desea que el servidor Web destino no sea el primero que se inscribió durante la instalación, siga estos pasos en la aplicación Meta4:

1. Acceda al Editor de parámetros mediante **Herramientas de desarrollo | Configuración de aplicación | Parámetros de aplicación**.
2. Seleccione la opción de menú **Herramientas | Parámetros avanzado**.
3. Desde el árbol de parámetros, seleccione la aplicación FORMS y la sección SINGLESIGNON y modifique los parámetros siguientes según el tipo de cambio que se quiera realizar.

Si el webserver es distinto:

1. Seleccione la clave WEBSERVER dentro de SINGLESIGNON.
2. Modifique los datos de la pestaña **Valores de ámbito**, introduciendo el nuevo valor que desea usar, es decir, *nombre de servidor*.

Si el puerto HTTP es distinto de 80:

-
1. Seleccione la clave PORT dentro de SINGLESIGNON.
 2. Modifique los datos de la pestaña **Valores de ámbito**, introduciendo el nuevo valor que desea usar, es decir, *número de puerto*.

Si el protocolo es HTTPS, es decir, si es un servidor Web con SSL:

1. Seleccione la clave PROTOCOL dentro de SINGLESIGNON
2. Modifique los datos de la pestaña **Valores de ámbito**, introduciendo el nuevo valor que desea usar, por ejemplo, *https*.



Solamente es posible configurar un servidor Web destino simultáneo.

Generación de claves pública/privada

Para hacer posible el funcionamiento de la identificación única del usuario, es necesario que el repositorio se firme con una clave privada aleatoria previamente a realizar la primera conexión con identificación única del usuario.

Para facilitar la instalación por primera vez del sitio Web, la primera vez que se intente acceder a la conexión Web con identificación única de usuarios en un repositorio de base de datos que no tenga firmado el repositorio, se realizará una firma aleatoria por defecto.

Si posteriormente se desea volver a firmar el repositorio, por ejemplo, porque se ha hecho una exportación del mismo y se desea asegurar que no puede establecerse una conexión con identificación única del usuario utilizando esta exportación desde otro usuario, puede utilizar la Presentación Por Defecto del Meta4Object de Conexión Unificada y ejecutar un método que volverá a firmar el repositorio, según se indica a continuación.

1. Acceda a la **Barra de tareas del desarrollador**. Por defecto se levanta presentaciones.
2. Introduzca SSC_ULOGON en el campo **Nombre** para ejecutar la presentación por defecto correspondiente de Meta4Object de Conexión Unificada.
3. Ejecute el botón que corresponde al método GENERATE_KEYS.

Configuración de un dominio de uno o varios repositorios con identificación única del usuario

Si desea que el cliente Windows origen realice una conexión con identificación única del usuario con un servidor Web conectado a través del servidor de aplicaciones a un repositorio diferente del primero, debe realizar una copia de

la clave privada del repositorio origen al repositorio destino.

Es necesario realizar una copia de los contenidos de las tablas M4RSC_ULOGON_PAS3 y M4RSC_ULOGON_PAS4 de un repositorio a otro. Para ello, siga estos pasos:

1. Inicie RAM-DL.
2. Conecte como origen el primer repositorio y como destino el segundo.
3. Seleccione la opción de menú **Tools | Galactic Query II**.
4. Ejecute el siguiente comando RDL:

```
Copy Data from Origin.M4RSC_ULOGON_PAS3 to
Destination.M4RSC_ULOGON_PAS3.

Copy Data from Origin.M4RSC_ULOGON_PAS4 to
Destination.M4RSC_ULOGON_PAS4.
```

Administración de autoridades certificadoras para Internet Explorer cuando el servidor Web destino es seguro

Si el servidor Web destino es seguro, para que pueda realizarse la navegación desde la opción de menú o presentación que invoca la conexión Web con identificación única del usuario, es necesario que la Autoridad Certificadora que emite el certificado SSL del servidor destino sea una de las autoridades en las que Internet Explorer confía.

Se debe hacer dos comprobaciones: que el certificado haya sido admitido y que la autoridad certificadora sea de confianza. Para ello, siga estos pasos:

1. Conecte con Internet Explorer al servidor Web seguro.
2. Haga clic en el botón **Seguridad SSL** en la barra de estado de la pantalla de Internet Explorer, para acceder a la pantalla **Certificado**.
3. Asegúrese en la pestaña **Ruta de certificación** de que la entrada en **Estado de certificación** indique *El certificado es válido*.
4. Compruebe en la pestaña **General** que el valor en **Emitido por** corresponde al nombre de la autoridad certificadora.
5. Acceda a **Herramientas | Opciones de Internet... | Contenido** y haga clic en el botón **Certificados....**
6. Verifique en la pestaña **Autoridades emisoras de certificados raíz en las que se confía** que la entrada en **Emitido para** coincide con la entrada de **Emitido por** del paso 4.

Configuración de la identificación única de usuario Web/ Web

Para hacer posible la conexión con identificación única del usuario Web/Web, es necesario configurar en el servidor Web origen la información del servidor Web destino.

Configuración del servidor Web destino

Para configurar el servidor Web o los servidores Web que se encargan de atender las peticiones de conexión con identificación única del usuario, edite el fichero de configuración **configclient.xml** por el siguiente procedimiento:

1. Abra el fichero **configclient.xml** con un editor.

EJEMPLO

Si el destino de la aplicación Web de Meta4 es C:\Program Files\Meta4\m4ws\peoplenet, edite el fichero C:\Program Files\Meta4\m4ws\peoplenet\classes\properties\configclient.xml.

-
2. Indique los proveedores de contenidos según el ejemplo que sigue.

Si su primer proveedor de contenido es **http://myserver.mydomain.com:myport** modifique el contenido del elemento URL_0 del siguiente modo:

```
<ExternalContentProvider>
  <LOCAL>/servlet/CrossPoster</LOCAL>
  <URL_0>http://myserver.mydomain.com:myport/servlet/SignOn</URL_0>
</ExternalContentProvider>
```

Si su segundo proveedor de contenido es **http://anotherserver.mydomain.com:someport**, añada un elemento URL_1 y edite el contenido del elemento como sigue:

```
<ExternalContentProvider>
  <LOCAL>/servlet/CrossPoster</LOCAL>
  <URL_0>http://myserver.mydomain.com:myport/servlet/SignOn</URL_0>
  <URL_1>http://anotherserver.mydomain.com:someport/servlet/SignOn</URL_1>
</ExternalContentProvider>
```

3. Cambie en el servidor Web origen, si es necesario, el nombre de la entidad que se hace cargo en el origen de empaquetar la información que se le envía al servidor Web destino. Para ello, edite el elemento LOCAL, si el agente que envía la información desde el servidor Web origen al destino no fuera el que se configura por defecto:

```
<ExternalContentProvider>
  <LOCAL>/anothercontext/servlet/CrossPoster</LOCAL>
  <URL_0>http://myserver.mydomain.com:myport/servlet/SignOn</URL_0>
</ExternalContentProvider>
```

Administración de certificados para servidor Web destino seguro

Si el servidor Web destino es seguro, es necesario realizar una exportación e importación de certificados en el servidor Web origen.

1. Conecte con Internet Explorer al servidor Web seguro.
2. Haga clic en el botón **SSL Secured** en la barra de estado de la pantalla de Internet Explorer, para acceder a la pantalla **Certificate**.
3. En la pestaña **Details** haga clic en el botón **Copy to File**.
4. Haga clic en el botón **Next** para continuar con el Certificate Export Manager Wizard.
5. Seleccione la opción **Base 64 Encoded X.509 (.cer)**
6. Indique el nombre del archivo, **C:\temp\venus-meta4-com.cer**.
7. Copie el archivo al servidor Web (el que se va a conectar al seguro) al entorno de ejecución y desarrollo de Java, es decir, al entorno de JDK del servidor Web origen.

```
cd C:\0\jdk1.3
mkdir stores
copy C:\temp\venus-meta4-com.cer C:\0\jdk1.3\stores
```

8. Ejecute la aplicación **keytool** desde el entorno JDK con los siguientes parámetros:

```
C:\0\jdk1.3\stores>..\bin\keytool -import -alias venus-meta4-com -
file venus-meta4-com.cer -keystore venus-meta4-com

Enter keystore password: changeit

Owner: EmailAddress=my_name@meta4.com, CN=venus.meta4.com, OU=the
wizard, O=merli

n, L=las rozas, ST=madrid, C=es

Issuer: EmailAddress=my_name@meta4.com, CN=venus.meta4.com, OU=the
wizard, O=merl

in, L=las rozas, ST=madrid, C=es

Serial number: 0
```

```
Valid from: Mon Sep 30 11:38:30 CEST 2002 until: Sat Jan 31
10:38:30 CET 2004
Certificate fingerprints:
MD5: 18:24:08:3E:1C:D8:03:35:15:FF:60:45:A7:29:1D:A1
SHA1: 30:C3:A9:C4:B3:8F:8C:DD:5A:4B:63:53:3E:AB:F4:0C:BD:34:87:04
Trust this certificate? [no]: yes
Certificate was added to keystore
```

9. Edite el `configclient.xml` para indicarle `TrustStore` y `TrustPassword`.

```
<ExternalContentProvider>
  <LOCAL>/servlet/CrossPoster</LOCAL>
  <TrustStore>c:/0/jdk1.3/stores/venus-meta4-com</TrustStore>
  <TrustPassword>changeit</TrustPassword>
  <URL_0>http://myserver.mydomain.com:myport/servlet/SignOn</URL_0>
</ExternalContentProvider>
```

Identificación única del usuario desde un portal Web externo

En esta sección se describen en detalle las componentes involucradas en la propuesta del establecimiento de una identificación única del usuario (Single Sign-On, SSO) desde un portal externo.

Para ello se explican los diferentes conceptos, pasos y procesos necesarios para llevar a cabo una conexión con identificación única del usuario, con una descripción de sus componentes y operaciones.

La identificación única del usuario afecta a los siguientes productos de Meta4:

- Aplicaciones Web para clientes HTML como SSE / SSM
- PeopleNet Rich Web



El cliente de desarrollo de Meta4 no está incluido en el sistema de identificación única del usuario aquí propuesto.

Existe una forma más evolucionada de realizar esta identificación única, mediante el uso de dominios de autenticación. Para consultar el nuevo modelo, dispone del manual *Single_Sign_On_Web.pdf*

A fin de poder entender el contenido de esta sección, es importante conocer los siguientes términos:

- **Contenedor Web:** aplicación que sirve páginas Web o aplicaciones Web. Por ejemplo portales, servidores de aplicaciones J2EE, servidores HTTP, etc.
- **Credencial:** elemento de seguridad que garantiza integridad, versión, fecha de validez, etc y permite a los sistemas de identificación electrónica validar el origen de una solicitud o petición.
- **J2EE:** estándar que especifica las componentes y procesos de servidores de aplicación Java y soportado por los grandes fabricantes.

Propuesta de identificación única del usuario con un portal externo

Un contenedor Web permite distintos mecanismos de autenticación en el momento en que un usuario solicita un recurso securizado.

Ejemplos de estos mecanismos pueden ser:

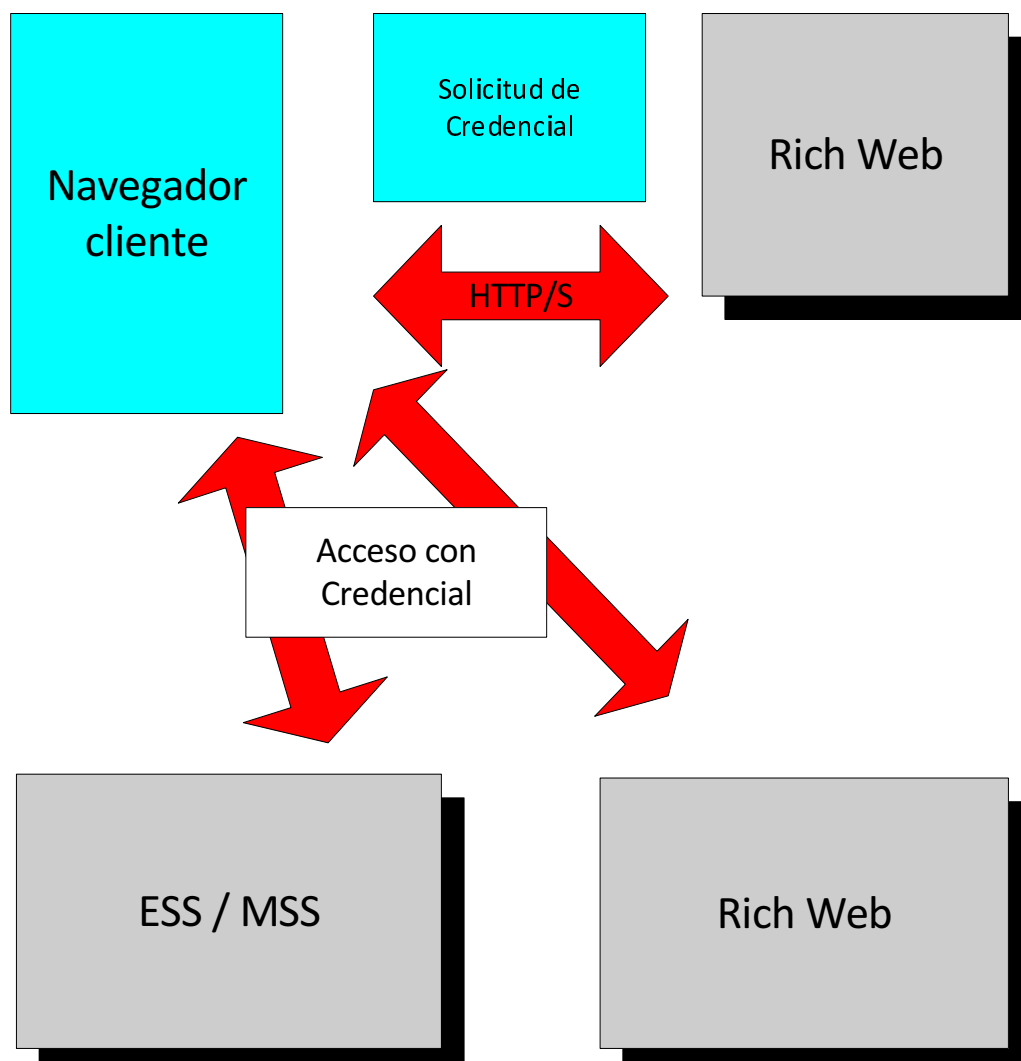
- Usuario / contraseña
- ID digital
- Conexión con identificación única del usuario con otros entornos (Kerberos, etc.)
- Específicos (NTLM Windows, etc.)

Para un contenedor Web que tenga una implementación Java (como J2EE) existe una interfaz especificada que permite la identificación unívoca de usuario autenticado. Esta interfaz está soportada por los grandes fabricantes (BEA, IBM, Oracle, Apache, etc.) y es en la práctica un estándar de autenticación para entornos J2EE.

La propuesta de Meta4 consiste en la delegación al contenedor Web de la autenticación del entorno PeopleNet. Para establecer dicha delegación existen distintos componentes de seguridad que se intercomunican basados en una credencial específica de Meta4.

Esta filosofía tiene como gran ventaja la desconexión entre la tecnología elegida para la autenticación en el contenedor Web y la tecnología adoptada por Meta4 para establecer la conexión con identificación única del usuario entre el contenedor y el entorno Meta4.

Esquema de componentes



Descripción de las componentes y operaciones

Navegador Cliente

El usuario final solicita el acceso a una de las aplicaciones de Meta4 por medio de un navegador cliente. El acceso se hace a través de una solicitud HTTP estándar. De esta manera no hay requerimientos específicos para el puesto

cliente dentro del esquema de identificación única del usuario propuesto por Meta4.

Contenedor Web

El contenedor es el responsable de la identificación del usuario entrante. Independientemente del esquema elegido para la autenticación, el contenedor se responsabiliza de inicializar la interfaz estándar J2EE para la identificación de usuario. En su defecto se podrá hacer una implementación específica en el contenedor que permita pasar la generación de la credencial para la identificación única del usuario a la infraestructura de Meta4.

Generador de Credenciales

Esta es la componente responsable de generar el elemento (credencial) que permitirá a las aplicaciones de Meta4 reconocer la autenticación recogida por el contenedor Web. Esta credencial será enviada al navegador cliente que efectuará una redirección a la aplicación de Meta4 que corresponda.

En caso de que no exista una situación de autenticación en el contenedor, se generará la redirección a la página de acceso habitual de la aplicación de destino.

Aplicaciones de Meta4: SSE/SSM, PeopleNet Rich Web

La aplicación concreta de Meta4 usa la credencial entrante para obtener varios datos relevantes durante el proceso de identificación única del usuario. Estos puntos son principalmente:

- Validez de la credencial entrante en lo concerniente a su integridad y fecha de validez
- Información del usuario que solicita la identificación única.

Generación de la credencial

Para que exista un contexto de seguridad durante el proceso de generación de la credencial hay una clave que permite cifrar la credencial. Esta clave está compartida entre el Generador de Credenciales y el Servidor de Aplicaciones de Meta4 en un certificado X.509 de 1024 bits. Para permitir el acceso de dicha clave al servidor de aplicaciones, está persistida y cifrada en el repositorio de BBDD y para su acceso por parte del Generador de Credenciales, está cifrada en disco.

La credencial es generada con una fecha de caducidad. Esta fecha de caducidad es parametrizable con el objetivo de minimizar las posibilidades de un uso no autorizado de la credencial.

Desarrollos particulares

Meta4 proporciona el API para la generación de credenciales y acceso a la infraestructura tecnológica. Sin embargo, es necesario un desarrollo particular para, entre otras cosas:

- Identificar el usuario que está logado en el portal
- Invocar la infraestructura proporcionada por Meta4.
- Enviar el resultado (HTML) al navegador cliente.

Requerimientos del software

Para el funcionamiento aquí descrito es necesaria la actualización tecnológica de las distintas aplicaciones de Meta4 además de la inclusión y configuración del Generador de Credenciales.

Puesta en marcha de la integración de la conexión con identificación única del usuario

En este apartado se describen los pasos necesarios para la puesta en marcha de la componente de integración de la conexión Web con identificación única del usuario de Meta4 sin dominios de autenticación.



Esta configuración se aplica solamente a los usuarios conectados con rol de aplicación M4ADM.

Generador del repositorio de claves

En el menú seleccione **Herramientas de desarrollo | Configuración de aplicación | Gestión de repositorio de claves**.

Se presentan dos opciones: **Generar claves (pública/privada) de firma y Exportar claves a fichero**. Si es la primera vez que accede a esta presentación, es recomendable la generación de claves. Una vez generadas las claves, ya no deben ser modificadas, ya que ello imposibilitaría la conexión de los clientes que actualmente estén utilizando la identificación única del usuario entre distintos repositorios de Meta4.



Tenga en cuenta que, aunque desde la opción **Gestión de repositorio de claves** es posible crear varios registros, el único registro utilizable a día de hoy sin dominios de autenticación es el contenido en el estándar con identificador "Meta4 Server Site = LOCAL".

Una vez disponibles las claves, estas pueden ser exportadas utilizando la opción **Exportar claves a fichero**. Se solicitarán 2 parámetros:

- Ruta para el nuevo repositorio de claves
- La clave que sella la clave privada en el repositorio.

El fichero generado está en formato PKCS12 y contiene un certificado X.509 con su clave pública y privada de 1024 bits. Estas claves también se quedan almacenadas en el repositorio de Meta4. Este fichero debe ser transportado al entorno Web de autenticación donde se generarán las credenciales.

Utilización del API del generador de credenciales

Librerías necesarias

Para el correcto funcionamiento del API de generación de credenciales es necesario que estén las siguientes librerías en el CLASSPATH de Java (según contenedor Web origen)

- m4common.jar
- bcbase64.jar
- log4j.jar
- jce-jdk13-119.jar

Estas librerías están disponibles en el CD del producto dentro de la carpeta *m4websso* en la componente *ToolKit*.



Si está utilizando la componente M4WEBSSO (librería m4common.jar), deberá actualizarla manualmente reemplazando los archivos jar existentes en su portal por los nuevos que se suministran en el CD. Consulte las KBs QA7416 (español) o QA7415 (inglés) para obtener instrucciones más precisas.

Invocación del API de generación de credenciales

La función Java que genera se llama `GenerateM4Credential()` y pertenece a la clase `com.meta4.common.m4cred.M4Credential`. Esta función espera como argumentos:

- La ruta al fichero de salida en formato PKCS12
- La clave para proteger la clave privada en el repositorio de claves
- El usuario para el cual se generará el certificado
- El idioma de conexión (reservado para uso futuro)
- El tiempo de expiración y el margen de error de sincronización de relojes entre la máquina que genera el certificado y la máquina donde se ejecutará el servidor de aplicaciones de Meta4 destino.

La función retorna una cadena representativa del certificado codificada en Base64.

Generación del HTML para la conexión con identificación única del usuario

Existen dos posibles productos de destino para la credencial: Rich Web y el SSE/SSM. Para cada uno de ellos existe un HTML específico que permite que el navegador cliente pueda conectarse al producto. Para generar dicho HTML existen dos funciones Java que reciben como entrada distintos parámetros entre ellos el certificado generado en el punto anterior. Pasamos a detallar cada una de estas funciones.

Conexión a Web con identificación única del usuario

La función a invocar es el método `generate ()` de la clase `com.meta4.common.m4credGenerateRedirectHTML`.

GenerateRedirectHTML.generate(sCredential, sUsr, sLang, sBaseURL, sServlet, sInitURI, sGotoURI);

Espera como parámetros de entrada:

- La credencial generada para el usuario en la función "GenerateM4Credential".
- El nombre del usuario autenticado en el contenedor Web
- El idioma para la conexión (ej. 2 para inglés, 3 para español, etc)
- La URL base para el servidor Web destino en el que se encuentra instalada la aplicación Web de Meta4 (protocolo, host, puerto)
- La ruta relativa para el servlet de login (normalmente `/servlet/login`).
- La ruta relativa (URI) para la página de inicialización de la aplicación Web. Esta ruta puede ser nula si no existe ninguna página que deba ser inicializada antes de llamar a la página destino.
- La ruta relativa para la página destino de identificación única del usuario.

Como resultado, esta función devuelve una cadena con el contenido HTML que debe ser enviado al navegador cliente. De ese modo el navegador efectúa una redirección a la aplicación Web.

El mecanismo de redirección no utiliza la tecnología de cookies pero es necesaria para mantener la sesión contra la aplicación Web de destino.

Conexión a Rich Web con identificación única del usuario

Función generate()

La función generate () en la clase com.meta4.common.m4credGenerateRedirectHTMLRich se encarga de producir el contenido HTML necesario para entrar en un entorno Rich Web.

GenerateRedirectHTMLRich.generate(sCredential, sUsr, sRole, sServer, sLang, sUpdateURL, null, null);

Esta función espera como parámetros de entrada:

- La credencial generada para el usuario en la función "GenerateM4Credential".
- El nombre del usuario autenticado en el contenedor Web.
- El rol del usuario conectado para la identificación única del usuario. Si se deja con valor "" se realiza la conexión con el rol por defecto, mientras que si se indica valor se realiza la conexión con dicho rol.
- El servidor de aplicaciones con el que se establecerá la conexión.
- El idioma de conexión (ej. 2 para inglés, 3 para español, etc.).
- La URL base para el servido Web en el que se encuentra el servicio de actualizaciones (protocolo, host, puerto). Así, por ejemplo, si la URL de entrada al Rich es: http://maquina:puerto/m4updateservices/m4richweb.html, en este parámetro se debe indicar tan solo la siguiente cadena: http://maquina:puerto/m4updateservices/.
- El nombre de la tarea que se desea invocar. Normalmente es nulo. Así, si el valor indicado es "", se entra en la presentación inicial del puesto cliente; en caso contrario se ejecuta la tarea indicada y no se muestra la ventana principal de la aplicación.
- Los parámetros para la tarea. Normalmente es nulo.

Como resultado esta función devuelve una cadena con el contenido HTML que debe ser enviado al navegador cliente. Como resultado el navegador efectúa una redirección a la aplicación Rich Web.

Función generateExtended()

La función `generateExtended ()` en la clase `com.meta4.common.m4credGenerateRedirectHTMLRich` se encarga de producir el contenido HTML necesario para entrar en un entorno Rich Web, permitiendo un comportamiento más detallado para el comportamiento de conexión.

`GenerateRedirectHTMLRich.generateExtended(sCredential, sUsr, sRole, sServer, sLang, sUpdateURL, null, null, oExtParams);`

Espera como parámetros de entrada:

- La credencial generada para el usuario en la función "GenerateM4Credential".
- El nombre del usuario autenticado en el contenedor Web.
- El rol del usuario conectado para la identificación única del usuario. Si se deja con valor "" se realiza la conexión con el rol por defecto, mientras que si se indica valor se realiza la conexión con dicho rol.
- Servidor de aplicaciones con el que se establecerá la conexión.
- El idioma para la conexión (ej. 2 para inglés, 3 para español, etc.).
- La URL base para el servidor Web en el que se encuentra el servicio de actualizaciones (protocolo, host, puerto). Así, por ejemplo, si la URL de entrada al Rich es `http://maquina:puerto/m4updateservices/m4richweb.html`, en este parámetro se debe indicar tan solo la siguiente cadena: `http://maquina:puerto/m4updateservices/`.
- El nombre de la tarea que se desea invocar. Puede ser nulo. Si el valor indicado es "" se entra en la presentación inicial del puesto cliente; en caso contrario se ejecuta la tarea indicada y no se muestra la ventana principal de la aplicación.
- Los parámetros para la tarea. Puede ser nulo.
- Y un objeto de parámetros extendidos de tipo `ExtendedParamsRich`. La clase `ExtendedParamsRich` contiene un método `setLogonConfig` que acepta valores particulares de la clase `LogonConfigRich`, que son los siguientes
 - `USE_PREVIOUS`: mantiene el comportamiento actual
 - `NONE`: indica a la función no ejecutar ninguna acción.
 - `ROLE_AND_PARAMS`: muestra la ventana de selección de roles y de configuración de parámetros de sesión.
 - `ROLE_ONLY`: muestra solamente la ventana de selección de roles .
 - `PARAMS_ONLY`: muestra solamente la ventana de configuración de parámetros de sesión.

Como resultado esta función devuelve una cadena con el contenido HTML que debe ser enviado al navegador cliente. Como resultado el navegador efectúa una redirección a la aplicación Rich Web.

Ejemplos

A continuación se presentan ejemplos para la conectividad con identificación única del usuario entre distintas aplicaciones destino, en los que solamente se ilustra cómo utilizar las funciones proporcionadas.

En caso de no poder verificarse la identidad del usuario conectado mediante la función `getUserPrincipal()`, se sugiere dar un error de seguridad en las trazas, para alertar de un posible acceso no autorizado.

Conexión con identificación única del usuario con aplicación Web

```

package com.meta4.test;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.meta4.common.m4cred.GenerateRedirectHTMLRich;import
com.meta4.common.m4cred.GenerateRedirectHTML;
import com.meta4.common.m4cred.M4Credential;
import com.meta4.common.m4cred.M4CredentialException;

/**
 * Servlet implementation class for Servlet: Test
 */
public class Test extends javax.servlet.http.HttpServlet implements javax.servlet.Servlet {
    private static final long serialVersionUID = 1L;
    public Test() {
        super();
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        M4Credential oCredential = new M4Credential();
        // gets the user id logged in the portal using standard API
        String sUser = request.getUserPrincipal().getName();
        try {
            // generates the credential
            String sCredential = oCredential.GenerateM4Credential("c:/temp/ca.p12",
"pass", sUser, "3", 600, 60);

```

```
                // ESS with destine and init page
                String sParsed = GenerateRedirectHTML.generate(sCredential, sUser, "3",
"http://testmachine:8100", "/servlet/login", "/servlet/CheckSecurity/JSP/tctools/tc_portal.jsp", "/servlet/
CheckSecurity/JSP/shco_js/shco_js_list_sched_tasks.jsp?zpag=shco_js/
shco_ap_wz_step_principal.jsp&znw=0");

                response.getOutputStream().println(sParsed);

            } catch (M4CredentialException e) {
                e.printStackTrace();
            }
        }
        protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
            doGet(request, response);
        }
    }
}
```

Conexión con identificación única del usuario con aplicación Rich Web:

En este ejemplo se utiliza la función generate() :

```
package com.meta4.test;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.meta4.common.m4cred.GenerateRedirectHTMLRich;
import com.meta4.common.m4cred.GenerateRedirectHTML;
import com.meta4.common.m4cred.M4Credential;
import com.meta4.common.m4cred.M4CredentialException;
/**
 * Servlet implementation class for Servlet: Test
 */
public class Test extends javax.servlet.http.HttpServlet implements javax.servlet.Servlet {
    private static final long serialVersionUID = 1L;
    public Test() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        M4Credential oCredential = new M4Credential();
        // gets the user id logged in the portal using standard API (produce exception if null)
        String sUser = request.getUserPrincipal().getName();
        try {
            // generates the credential
            String sCredential = oCredential.GenerateM4Credential("c:/temp/ca.p12",
"pass", sUser, "3", 600, 60);

            // Rich web without task id
            String sParsed = GenerateRedirectHTMLRich.generate(sCredential, sUser, "",
"testmachine.meta4.com", "3", "http://testmachine.meta4.com/virtualdir/", null, null);

            //

```

Rich web with task id

```
//          String sParsed = GenerateRedirectHTMLRich.generate(sCredential, sUser, "",
"testmachine.meta4.com", "3", "http://testmachine.meta4.com/virtualdir/", "MN_TaskBar", null);

        response.getOutputStream().println(sParsed);
    } catch (M4CredentialException e) {
        e.printStackTrace();
    }
}

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    doGet(request, response);
}
```

En este ejemplo se utiliza la función generateExtended() :

```
package com.meta4.test;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import com.meta4.common.m4cred.GenerateRedirectHTMLRich;
import com.meta4.common.m4cred.GenerateRedirectHTML;
import com.meta4.common.m4cred.M4Credential;
import com.meta4.common.m4cred.M4CredentialException;
/**
 * Servlet implementation class for Servlet: Test
 */
public class Test extends javax.servlet.http.HttpServlet implements javax.servlet.Servlet {
    private static final long serialVersionUID = 1L;
    public Test() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        M4Credential oCredential = new M4Credential();
        // gets the user id logged in the portal using standard API (produce exception if null)
        String sUser = request.getUserPrincipal().getName();
        try {
            // generates the credential
            String sCredential = oCredential.GenerateM4Credential("c:/temp/ca.p12",
"pass", sUser, "3", 600, 60);

            // Rich web without task id
            String sParsed = GenerateRedirectHTMLRich.generate(sCredential, sUser, "",
"testmachine.meta4.com", "3", "http://testmachine.meta4.com/virtualdir/", null, null);

            //
```

```

        // Rich web with extended params
        ExtendedParamsRich extendedParams = new ExtendedParamsRich();
        extendedParams.setLogonConfig(LogonConfigRich.ROLE_ONLY);
        String sParsed =
GenerateRedirectHTMLRich.generateExtended(sCredential, sUser, "", " testmachine.meta4.com", "3",
"http://testmachine/virtualdir/", null,null, extendedParams);

        response.getOutputStream().println(sParsed);
    } catch (M4CredentialException e) {
        e.printStackTrace();
    }
}

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    doGet(request, response);
}

```

Configuración de la autorización en servicios Web

Introducción a la autorización

En las aplicaciones de Meta4, la autorización se basa en los conceptos de rol y organización. El concepto de organización permite que varios clientes compartan una misma base de datos, y el concepto de rol permite manejar la seguridad dentro de un mismo cliente.

El usuario de las aplicaciones de Meta4 tiene un rol por defecto, una lista posible de roles, y esos roles están asociados a sociedades. También tiene el usuario de aplicación asociada una sociedad por defecto.

Cuando el usuario entra en una aplicación Web para cliente HTML, entra con el rol por defecto y la organización por defecto, a menos que se desarrollen unas páginas que específicamente le permitan durante la conexión elegir cuál será el rol y/o la sociedad con la que entrará en la aplicación.

Existen algunas aplicaciones Web para clientes HTML que requieren un rol por defecto distinto del que se establece para el mismo usuario en un entorno Rich Web. Para solucionar este problema se ha definido el concepto de "rol por defecto para las aplicaciones Web".

Configuración del rol por defecto para las aplicaciones Web

Para configurar el rol por defecto de las aplicaciones Web, se tiene que marcar uno o varios roles como "Roles del Portal", y asegurarse de que un usuario tiene uno, y solamente uno, de estos roles.

Para marcar un rol como "Rol del Portal", desde el editor de roles de aplicación (Herramientas de desarrollo | Seguridad | Roles de aplicación) seleccione el primer rol que desea establecer como rol por defecto. A continuación acceda a **Opciones de programa** y seleccione la opción **Cambiar rol por defecto** (SWITCH_ROLE) de la aplicación **Parámetros de cliente ligero** (THINCLIENT). Marque la casilla **Sujeto a seguridad** (sólo lo tendrá que realizar la primera vez si no está ya marcada) y finalmente marque la casilla **Permiso por defecto** y pulse **Aceptar**. De esta forma, tendrá establecido un primer rol por defecto, que será el que utilice el sistema cuando el usuario se conecte a la aplicación Web.

Para establecer otro "Rol del Portal", basta con seleccionar otro rol, acceder a **Opciones de programa** y marcar la casilla **Permiso por defecto**.

Si al realizarse la conexión a la aplicación Web del cliente HTML el usuario no tiene ningún rol marcado como rol del portal, el sistema asignará el rol por defecto normal, el mismo que se utiliza para el cliente Rich Web.

En caso de que un usuario tuviera en su lista de roles más de un rol del portal, el sistema también asignará el rol por defecto normal. Esto puede ser un problema si la aplicación se ha diseñado con filtros de seguridad que dependen de los roles del portal.

Personalización de recursos por sociedad

Las aplicaciones Web para clientes HTML de Meta4 pueden ser explotadas en entornos multicliente. En dichos entornos, un solo conjunto de páginas puede dar servicio a múltiples clientes.

En esta sección se explicarán los conceptos necesarios para personalizar recursos dentro de aplicaciones Web HTML (como SSE/SSM) de modo que estas aplicaciones puedan ser diferentes para múltiples clientes, sin necesidad de tener múltiples entornos para ellos. De este modo, se explicará:

- Cómo personalizar ficheros de la aplicación Web: páginas JSP, ficheros properties, CSS, imágenes, etc. de modo que para un cliente del SSE/SSM se pueda especificar una imagen, página JSP, etc., diferente a la que se distribuye con el producto estándar.
- Cómo cambiar literales de las páginas (tanto los datos de la página como las etiquetas o nombres de ítems y nodos). También se explica la forma de parametrizar esta funcionalidad y los literales susceptibles de ser cambiados.



La funcionalidad de sustitución de literales debe usarse con precaución, sólo para cambios legales imprescindibles como el de "Sociedad" por "Organismo", puesto que su uso indiscriminado podría afectar al funcionamiento de la aplicación.

Determinar el nodo del árbol de ejecución para la sociedad a personalizar

Cuando un usuario se conecta a la aplicación PeopleNet, lo hace dentro de una sociedad determinada. Según la sociedad, se sitúa dentro de un nodo del árbol de ejecución, una estructura de datos destinada a permitir la personalización de elementos de la tecnología de Meta4.

En el árbol de ejecución, cada nodo hoja (no contenedor de otros nodos) se corresponde con una sociedad. Un nodo hoja puede no tener asignada una sociedad, pero siempre que se busque una sociedad tiene que aparecer un nodo hoja. Por tanto, cuando un usuario se conecta a la aplicación Web, siempre se podrá determinar el valor del identificador de su nodo del árbol de ejecución (NAE).

Supongamos que deseamos personalizar los recursos para la sociedad de identificador 1000. En el árbol de ejecución (Herramientas de desarrollo | Explorador de proyectos) encontramos el nodo hoja que tiene asociada esta sociedad. En nuestro ejemplo, supongamos que el identificador es DATOS.

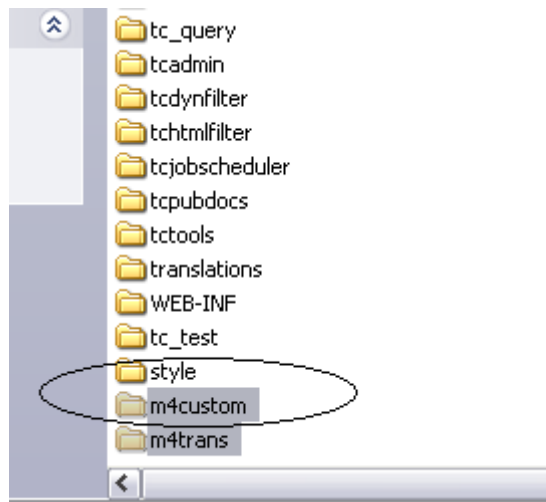
Crear el directorio de personalización (m4custom)

La funcionalidad de personalización se activa por configuración (está desactivada por defecto) y presenta particularidades según el tipo de elemento a personalizar. La lista de elementos personalizables es la siguiente:

- Personalizar página JSP de datos
- Personalizar fichero properties
- Personalizar otros ficheros
- Personalizar hojas de estilo CSS
- Personalizar página JSP de login

Dada la estructura de directorios de una aplicación de cliente HTML como el SSE/SSM, para proceder a la personalización de páginas debe existir un directorio de personalización (directorio m4custom), situado al mismo nivel que todos los directorios de tareas de cliente ligero (es decir, <raíz-aplicación-web>/m4custom).

Figura 2. Directorio m4custom de personalización de páginas



Activar el mecanismo

Verifique, en primer lugar, que está activado el mecanismo de personalización y sustitución. Esto se realiza desde el fichero web.xml

EJEMPLO

```
<filter>
  <filter-name>M4RedirectFilter</filter-name>
  <filter-class>com.meta4.filters.redirect.M4RedirectFilter</filter-
class>
  <init-param>
    <param-name>enable</param-name>
    <param-value>true</param-value>
  </init-param>
</filter>

<filter-mapping>
  <filter-name>M4RedirectFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

Personalización de páginas JSP

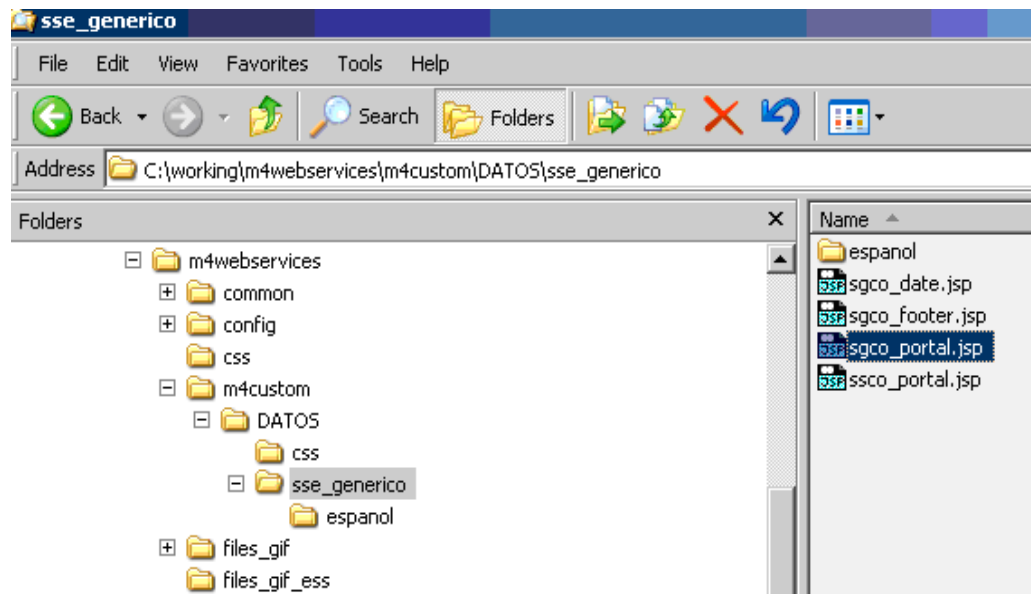
Como hemos dicho anteriormente, la personalización de las páginas se lleva a cabo atendiendo a la sociedad. Supongamos que deseamos que el usuario (con sociedad 1000 e identificador de nodo de árbol de ejecución DATOS) vea una página de portal personalizada y no la estándar. Por ejemplo:

`http://sistema:puerto/servlet/CheckSecurity/JSP/sse_generico/sgco_portal.jsp`

Para personalizar esta página procederemos del siguiente modo:

1. Encontrar el directorio de personalización <raíz-aplicación-web>/m4custom
2. Crear una carpeta con el mismo identificador del nodo del árbol de ejecución, es decir: <raíz-aplicación-web>/m4custom/DATOS
3. Crear una copia de la página que queremos personalizar bajo el directorio de personalización, respetando la estructura de directorios original:
 - La página estándar estaba en `sse_generico/<idioma>/sgco_portal.jsp`.
 - La nueva página personalizada tendrá que copiarse a la carpeta `m4custom/DATOS/sse_generico/<idioma>/sgco_portal.jsp`.

Figura 3. Directorio de personalización de páginas



4. Tratamiento de los includes: Si se desea personalizar una página JSP, hay que tener la precaución de comprobar si incluye (usando la directiva "include") otras páginas. Para que la página siga funcionando existen dos opciones:

-
- Copiar en la estructura m4custom todas las páginas que se referencian desde la página personalizada. Esta opción será conveniente cuando se quieran personalizar también las páginas incluidas.
 - Modificar la página personalizada para que utilice las páginas incluidas estándar. Si los includes se han realizado con rutas relativas, este cambio debe hacerse teniendo en cuenta que la nueva página ha descendido dos niveles en la jerarquía de directorios (en el caso de haberse utilizado rutas absolutas, no sería necesario ningún cambio).

EJEMPLO

Por ejemplo, el include:

```
<%@ include file="sse_generico_taglib.jsp" %>
```

Podría cambiarse por:

```
<%@ include file="../../../sse_generico_taglib.jsp" %>
```

Con este proceso lo que se consigue es que la aplicación cargue la página personalizada sgco_portal.jsp.

Este proceso sólo se aplica para la sociedad DATOS (1000). Si se logara en el SSE/SSM un usuario cuyo rol tuviera asociada la sociedad 0001, se cargaría la página original.

Es importante decir que en la sobrescritura de páginas jsp se aplica una especie de herencia de la siguiente manera. Si, en vez de crear un directorio hijo de m4custom denominado DATOS, se llamase CLIENT, es decir, como el nodo padre de DATOS en el árbol de proyectos, los cambios de la jsp se aplicarían a todas las sociedades que cuelgan de CLIENT, incluido el nodo de nuestro ejemplo denominado DATOS.

La redirección de JSP la realiza, cumpliendo todos los requisitos de seguridad de datos, el **servlet CheckSecurity**. Este servlet garantiza que no se puede acceder indebidamente tampoco a la página personalizada si la seguridad de la tarea lo establece.

Personalización de archivos .properties

Supongamos que deseamos que el usuario comentado anteriormente (con sociedad 1000 e identificador de nodo de árbol de ejecución DATOS) vea un fichero de literales estáticos (.properties) propio de esta sociedad.

Para eso dejaremos de utilizar la clase `Properties` de `java.util.*`, pasando a utilizar la clase `M4PropertiesRedirect`, que se encargará de seleccionar el fichero de literales adecuado.

EJEMPLO

Supongamos que una página que carga literales (`sse_generico/`
`sse_generico_trans.jsp`) está programada, por ejemplo, del siguiente modo:

```
/* Properties object */
java.util.Properties Tran = new Properties();
Tran.load(application.getResourceAsStream("/translations/
ess_mss_gen_" + sLangEss + ".properties"));
```

Ahora hay que realizar el siguiente cambio:

- Copiar la página al directorio de personalización (`m4custom/DATOS`)
- Realizar el siguiente cambio de código:

```
/* Properties object */
com.meta4.redirect.M4PropertiesRedirect Tran = new
com.meta4.redirect.M4PropertiesRedirect();
Tran.load(pageContext, "/translations/ess_mss_gen_" + sLangEss + ".properties");
```

En el caso de los ficheros `.properties`, si se sobrescribe alguno de ellos en una sociedad padre `CLIENT`, y asimismo se sobrescribe en el nodo `DATOS`, se produce una acumulación, es decir, se suman las propiedades del hijo a las del padre y las propiedades que se encuentran repetidas, se sobrescriben con los valores especificados en el nodo inferior (`DATOS`).

Personalización de otros archivos (imágenes, javascript, etc.)

Supongamos que deseamos que el usuario comentado anteriormente (con sociedad 1000 e identificador de nodo de árbol de ejecución `DATOS`) vea ficheros de imágenes diferentes a los que proporciona el producto estándar.

Este proceso es similar al anterior: se debe copiar la imagen nueva dentro de la carpeta `m4custom/DATOS/<ruta-imagen>`.

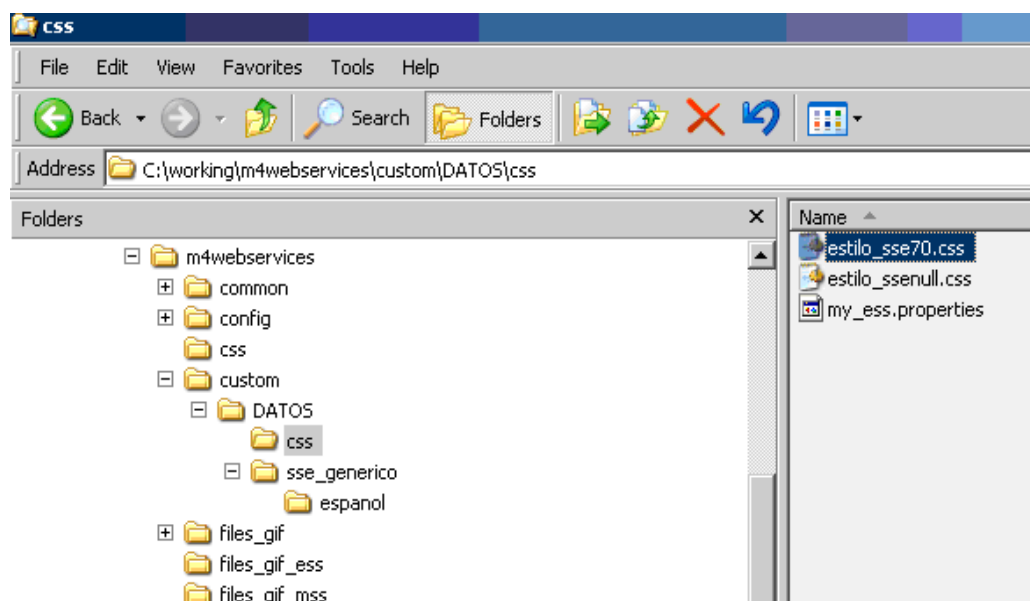
La personalización, en lugar de realizarla el servlet CheckSecurity, la lleva a cabo un filtro denominado **M4RedirectFilter**, presente en web.xml, que debe estar activo.

Personalización de las hojas de estilo (CSS)

Además de lo comentado en el apartado anterior, si se desean personalizar hojas de estilo (CSS) el proceso tiene algunas particularidades que se explican a continuación.

En nuestro ejemplo, vamos a personalizar la hoja de estilo del portal del SSE, un fichero denominado estilo_sse70.css y la hoja de estilos para el portafirmas, denominada estilo_sse100.css. Estos dos ficheros se encuentran dentro de la aplicación estándar en el directorio css. La estructura de directorios creada para personalizarlos es la siguiente:

Figura 4. Directorio de personalización de las hojas de estilo (CSS)



Imaginemos que queremos parametrizar el color de los enlaces visitados de la página del portal, correspondiente al fichero estilo_sse70.css. Para ello se deben insertar las siguientes líneas en este fichero:

```
<m4:def file = "/css/my_ess.properties" alias = "A" />
A:link
{
    BORDER-RIGHT: #395a82;
    BORDER-TOP: #395a82;
    BORDER-LEFT: #395a82;
```

```

        COLOR: #15314c;
        BORDER-BOTTOM: #395a82;
        BACKGROUND-COLOR: transparent;
        TEXT-DECORATION: none
    }
    A:visited
    {
        COLOR: <m4:value alias = "A" property = "ev_ess.visited"/>;
        BACKGROUND-COLOR: transparent;
        TEXT-DECORATION: none
    }

```

La primera línea especifica que en el fichero de propiedades situado en la ruta /css/my_ess.properties se encuentran una serie de nombres=valor que representan propiedades de la hoja de estilo que se quieren sobrescribir. Por ejemplo: ev_ess.visited=magenta.

La segunda línea especifica cómo se cambia el valor del COLOR de A.visited apuntando al valor de una propiedad del fichero my_ess.properties. En nuestro caso, ev_ess.visited.

En este ejemplo, queremos sustituir el color de la propiedad visited de la hoja de estilo por el magenta.

También se ha parametrizado el título de la parte de portafirmas de forma que cambie el color y aparezca en verde. Esto se hace de igual forma que en el caso anterior. Editamos el fichero estilo_essnull.css y se añaden las siguientes líneas:

```

<m4:def file = "/css/my_ess.properties" alias = "A" />
A:link
{
    BORDER-RIGHT: #395a82;
    BORDER-TOP: #395a82;
    BORDER-LEFT: #395a82;
    COLOR: #15314c;
    BORDER-BOTTOM: #395a82;
    BACKGROUND-COLOR: transparent;
    TEXT-DECORATION: none
}

.....

.titulofuncional
{

```

```

    FONT-SIZE: 11px;
    FONT-WEIGHT: bolder;
    COLOR: <m4:value alias = "A" property = "ev_ess.titulofuncional"/
>;
    text-indent: 10px;
    background-color:#610000;
    background-image: url(/iconos/back_title_task.png);
    background-repeat: repeat-x;
    height: 20px;
}

```

Personalizar una página JSP previa a la conexión

Supongamos que deseamos que el usuario comentado anteriormente (con sociedad 1000 e identificador del nodo de árbol de ejecución DATOS) vea una página de login personalizada. Es decir, que deseamos que en lugar de ver la página de login estándar, vea una especial para él:

http://sistema:puerto/jobsite_g/espanol/generico_login.jsp

En este caso, como el usuario no está conectado, el sistema no tiene información de su conexión para determinar cuál será el nodo del árbol de ejecución que se aplica para él. La información que se utilizará en este caso es el nombre cualificado de host (host.dominio) que el usuario ha empleado para la conexión.

Por ejemplo, si el cliente de la sociedad especificada se conecta a través del nombre cualificado de host datos.meta4.com, se debe comprobar que en la presentación por defecto del Meta4Object SRTC_DNS_NODE_MAP se ha creado un nuevo registro en el que figure dicha correspondencia.

Una vez que está definido que si se accede a **datos.meta4.com**, el nodo del árbol de ejecución correspondiente al usuario es automáticamente DATOS, para personalizar la página de login, sólo nos queda copiar, como en el ejemplo anterior, la página de login al directorio de personalización.

Insistiendo en el procedimiento, para personalizar esta página procederemos así:

- Localizar el directorio de personalización <raiz-aplicación-web>/m4custom y la carpeta con el mismo identificador del nodo del árbol de ejecución, es decir: <raiz-aplicación-web>/m4custom/DATOS.
- Crear una copia de la página de login que queremos personalizar bajo el directorio de personalización, respetando la estructura de directorios original.

- La página estándar estaba en `jobsite_g/<idioma>/generico_login.jsp`.
- La nueva página personalizada tendrá que copiarse a la carpeta `m4custom/DATOS/jobsite/<idioma>/generico_login.jsp`.
- Tratamiento de los includes: como se ha mencionado en la sección anterior, se debe verificar si la página personalizada de login tiene dependencias, trasladándolas a la nueva ubicación si es preciso. Es decir, si la página a personalizar incluye otras páginas JSP, se tienen dos posibilidades:
 - Replicar estas páginas en la nueva estructura de directorios.
 - Cambiar las rutas de los includes de las páginas atendiendo a la nueva estructura de directorios personalizada, según se comentó en secciones anteriores.

Configuración de la sustitución de literales

Se pueden realizar varios tipos de sustitución de literales en aplicaciones Web de cliente ligero HTML:

- Sustitución de metadatos (nombres de ítems o nodos)
- Sustitución de datos en páginas.
- Sustitución de datos en ítems de sólo lectura

Si se activa la funcionalidad de cambio de literales, todas las páginas del producto serán procesadas para generar unas nuevas con los literales traducidos (aparecerán en la carpeta `m4trans`). Asimismo, se realiza un proceso para reemplazar los includes de las páginas JSP y ponerlos en formato ruta absoluta. De esta forma, la página traducida quedará coherente con los includes; si no, las páginas incluidas no se encontrarían en la ruta original.

Esta funcionalidad convive con la funcionalidad para personalizar páginas. De esta forma nos podríamos encontrar con una estructura de directorios en la que coexistan páginas traducidas no personalizadas por el usuario y páginas traducidas personalizadas por el usuario.

Sustitución de metadatos

Este apartado explica cómo se puede realizar el proceso de sustitución de metadatos en aplicaciones de cliente ligero. Los metadatos que se contemplan son nombres de ítems y nombres de nodos.

Para activar esta funcionalidad se debe parametrizar desde PeopleNet de la siguiente manera:

-
- Abra el menú Configuración de la aplicación / Parámetros de aplicación. Desde ahí, haga clic en Parámetros Avanzados. Una vez dentro, sitúese en la sección FORMS/CHANGETEXT. Esta sección tiene una clave denominada ENABLE, mediante la cual se puede activar la sustitución de literales. Esta clave se debe parametrizar por usuario de aplicación. Si tiene valor 1, se activa la funcionalidad de sustituir literales y si está a cero, no está activada para este usuario.
 - El resto de claves que se encuentran bajo la rama CHANGETEXT son las palabras que se quieren sustituir en la aplicación. Por cada una de estas claves se pueden tener una serie de nombre=nombre_sustituído que se aplicarán en cada caso.

Sustitución de datos en páginas

Esta funcionalidad se refiere a la posibilidad de sustituir literales dentro de ciertos ficheros (páginas JSP, ficheros JavaScript) de las aplicaciones Web para clientes HTML. Por defecto se realiza la traducción sobre ficheros con extensiones JSP y JS.

Sustitución de datos en ítems de solo lectura

Al igual que para los metadatos (nombres de ítems o nodos), puede ser necesario aplicar cambios de literales según la organización del cliente. En las aplicaciones Web HTML, como SSE/SSM, algunas opciones asimiladas a metadatos, como los elementos del menú, se traen como datos.

Para esto, se ha añadido al tag <m4:item> una opción para estos casos (en los que, como en los menús del SSE, tiene que hacerse la sustitución de literales). Dicha opción es replaceliterals.

Un ejemplo de utilización:

```
<m4:item outputdef="ROOTNODE" item="STD_COMMENT"
replaceliterals="true"/>.
```

Pruebas para medir la disponibilidad del servicio

Para medir la disponibilidad de los servicios proporcionados por las aplicaciones de Meta4, se han desarrollado una serie de pruebas, englobadas bajo el nombre **m4sitewatcher**, que consisten en peticiones de conexión o semánticas a distintos elementos de la aplicación de Meta4.

Instalación como aplicación independiente

Para instalar las pruebas que abarca m4sitewatcher, simplemente desempaquete en un directorio el contenido de
<CD>\m4toolkit\m4sitewatcher\m4sitewatcher.zip.

EJEMPLO

En Windows, si se desempaquetaran las pruebas en C:\, la estructura de directorios quedaría así

```
C:\m4sitewatcher\app\classes\config
C:\m4sitewatcher\app\classes\config\m4sitewatcherconfig.xml
C:\m4sitewatcher\app\classes\properties
C:\m4sitewatcher\app\classes\properties\m4logsystem.cfg
C:\m4sitewatcher\app\lib\3rdparty-all.jar
C:\m4sitewatcher\app\lib\m4common.jar
C:\m4sitewatcher\app\lib\m4communication.jar
C:\m4sitewatcher\app\lib\m4connect.jar
C:\m4sitewatcher\app\lib\m4properties.jar
C:\m4sitewatcher\app\lib\m4sitewatcher.jar
C:\m4sitewatcher\app\lib\m4thcl.jar
C:\m4sitewatcher\app\lib\snmp4j-1.11.1.jar
```

Donde:

- **m4sitewatcherconfig.xml**: es el fichero de configuración de las pruebas.
- **m4logsystem.cfg**: es el fichero de configuración de las trazas.



La configuración de las trazas es análoga a la descrita para los servicios Web de Meta4, pues el sistema de trazas de las aplicaciones de prueba está basada en "log4j".

Configuración

En el fichero **m4sitewatcherconfig.xml** aparece una entrada por cada notificador (notifier) o prueba a ejecutar.

```
<?xml version="1.0" encoding="UTF-8"?>
<m4sitewatcher id="default" workers="3">
[...]
<notifiers>
[...]
<element id="XXN" delay="xx"
        class="com.meta4.sitewatcher.notifiers.M4XXNNotifier">
    <config dispatcher="localhost" port="3102"/>
</element >
[...]
</notifiers>
</m4sitewatcher>
```

En el elemento raíz <m4sitewatcher> se incluye un identificador (id="default") y un número configurable que normalmente no es necesario alterar.

Cada prueba individual viene identificada con:

- Identificador del notificador (id="XXN"): un identificador único dentro del fichero
- Clase Java que implementa el notificador (class), con el paquete com.meta4.sitewatcher.notifiers, subpaquete identificador del notificador, y nombre de la clase.
- Configuración individual del test (por ejemplo: <config dispatcher="localhost" port="3102"/>). Cada test necesitará una configuración especial, que se describe en las próximas secciones.
- La periodicidad de ejecución de la prueba (delay="xx") no se utiliza al ser un agente externo el que va a lanzar la herramienta.

La configuración de las pruebas que se distribuyen constituye un ejemplo de configuración, es decir, el fichero XML debe adaptarse obligatoriamente a la topología del sitio a monitorizar (indicando puertos, nombres de host, número de tests a ejecutar, etc.).

Pruebas disponibles

Los notificadores o pruebas concretas del site disponibles en la aplicación son los siguientes:

- Tests de verificación del funcionamiento de elementos individuales del site:
 - **M4SocketConnectNotifier**: conexión TCP al puerto indicado utilizando el API de java.net.
 - **M4DispatcherConnectNotifier**: envío de paquete de datos de conexión directa al dispatcher.
 - **M4XMLLoginNotifier**: conexión de cliente ligero (M4XML) al servidor de aplicaciones o dispatcher que se indica (prueba de servicio XML).
 - **M4SNMPServerNotifier**: consulta a una propiedad SNMP de un servidor que tenga activada la configuración de SNMP versión 2.
- Tests de verificación del funcionamiento del estado general del servicio:
 - **M4LoginServletNotifier**: conexión a la aplicación Web para clientes HTML, utilizando el servlet de login.
 - **M4GatewayConnectNotifier**: envío de paquete de datos de conexión a la pasarela.
 - **M4DownloadNotifier**: conexión HTTP a un servidor Web y descarga de un archivo (verifica que el servicio de archivos Update Services funciona).

Pruebas de funcionamiento de elementos individuales

M4SocketConnectNotifier

Descripción

Esta prueba realiza una conexión TCP al puerto indicado, utilizando el API de java.net.

Configuración

<config address="localhost" port="8100" /> donde address es el nombre cualificado del host y port el puerto TCP.

Conexión

A través del constructor del objeto M4SocketConnector, que es el que se utiliza en las comunicaciones del cliente ligero.

M4DispatcherConnectNotifier

Descripción

Esta prueba envía un paquete de datos de conexión directa al dispatcher.

Configuración

<config dispatcher="localhost" port="3102" />, donde dispatcher es el nombre cualificado del host y port el puerto TCP.

Conexión

Se realiza a través del API de M4Operations connectRequest(), diseñado para calcular el servidor y puerto candidato.

M4XMLLoginNotifier

Descripción

Esta prueba realiza una conexión de cliente ligero al servidor de aplicaciones o dispatcher que se indica (prueba de servicio XML).

Configuración

<config server="localhost" port="3002" user="PT_GUEST" pass="PT_GUEST" lang="3"/>, donde server es el nombre cualificado del host, port el puerto TCP, user y pass, el usuario y contraseña que se utilizarán para la conexión, y en lang aparece el idioma de conexión.

Conexión

Se realiza a través del API login() de M4Operations().

M4SNMPServerNotifier

Descripción

Esta prueba realiza una consulta a una propiedad SNMP de un servidor que tenga activada la configuración de SNMP versión 2 (que startup.obl tenga el subsistema y el estado V1V2C activados, es decir, en TYPE = "SNMP" tener ACTIVE = "TRUE" y SNMP_V1V2C_STATE = "ENABLED", siendo el puerto de SNMP por defecto SNMP_PORT = 3001).

Configuración

<config command="get" oidroot="1.3.6.1.4.1.19883.3.1.1.1" oiddetail=".1.9.0" snmpid="memorylevel" snmpserver="localhost" snmpport="3001" />, donde snmpserver y snmpport son el nombre del server y su puerto de SNMP, oidroot es la raíz de las OIDs aplicables al servidor de aplicaciones de Meta4 (recordando que el prefijo iso.org.dod.internet.private.enterprise es 1.3.6.1.4.1, el número de Meta4 es 19883 y el servidor de aplicaciones es 3.1.1.1), oiddetail el identificador del valor a consultar (por ejemplo 1.9.0) y snmpid es una cadena que permita identificar dicho valor (por ejemplo, memorylevel).

Se han incluido por defecto los siguientes ejemplos de SNMP GET (consultar

fichero snmp.xml para indagar más tests que hacer):

```
memorylevel (1.9.0) -> normal, alert, critical
consumed_system_memory (1.501.0) -> MB sistema
cpu_usage (1.502.0) -> %
m4server_memory_countability (1.503.0) -> MB proc
job_scheduler_executor_service.exited_executors -> 5.1007.1001.5.0
```

Conexión

Se realiza a través de un comando GET de SNMP implementado con el API snmp4j.

Comentarios

Debe estar activada la funcionalidad SNMP v2 del servidor de aplicaciones.

Pruebas de estado general del servicio

M4LoginServletNotifier

Descripción

Esta prueba realiza una conexión en la aplicación Web para clientes HTML, utilizando el /servlet/login.

Configuración

<config user="PT_GUEST" pass="PT_GUEST" lang="3" url="/servlet/CheckSecurity/JSP/reports/reports.jsp" host="localhost" port="8100" loginuri="/servlet/login" logouturi="/servlet/login?_LOGOUT"/>, donde user y pass son las credenciales de conexión, host y port la dirección de la aplicación Web, loginuri ruta del servlet de login, logouturi la cadena que define la petición de logout, y url la página a la que nos llevaría el servlet en caso de éxito.

Conexión

Se realiza a través de un POST via java.net.URLConnection.

Comentarios

No se puede utilizar la página de logout como URL de logout.

M4GatewayConnectNotifier

Descripción

Esta prueba realiza una conexión análoga a la que realizan los clientes Rich Web cuando envían sus peticiones a través de la pasarela HTTP.

Configuración

<config gateway="localhost" port="8100" serverhost="localhost" serverport="3002"/>, donde gateway y port son la dirección del gateway, serverhost y server port los identificadores del dispatcher y/o servidor al que dirigir la petición por detrás.

Conexión

Se realiza mediante un API especial de m4communication.

M4DownloadNotifier

Descripción

Esta prueba se conecta a un servidor Web y se descarga un fichero. Sirve para comprobar la disponibilidad del servidor Web de Update Services, u otro.

Configuración

<config host="localhost" port="8100" fileuri="/M4WizardIn.reg"/>, donde host y port son la dirección del servidor Web, y el fichero a descargar debe ser especificado en el campo fileuri.

Conexión

A través de HttpURLConnection.

Lanzamiento de las pruebas

Para lanzar un test individual se utiliza la clase M4ExecuteNotifier, pasándole como argumento el identificador del test que vamos a lanzar.

Si, por ejemplo, se quiere lanzar el test M4GatewayConnectNotifier1 que se va a conectar a un entorno con las siguientes características: el Gateway está en http://localhost:8100 y el servidor de aplicaciones en localhost:3002. Entonces:

EJEMPLO

```
<element id="M4GatewayConnectNotifier"
class="com.meta4.sitewatcher.notifiers.M4GatewayConnectNotifier">
  <config gateway="localhost" port="3102" serverhost="localhost"
serverport="localhost" user="M4ADM" pass="M4ADM" lang="3"
usessl="false"/>
</element>
```

Para lanzar una sola prueba de estas características, podríamos escribir un fichero .bat que lanzara la clase del notificador, estableciendo el entorno de librerías Java necesarias.

EJEMPLO

En un sistema Windows, para lanzar un solo test, previamente configurado, se podría escribir un archivo tipo .bat, por ejemplo, C:\m4sitewatcher\app\test-notifier.bat, con una sintaxis como la siguiente:

```
set SITEWATCH_BASE_DIR=C:\m4sitewatcher\app

set SITEWATCH_CLS_DIR=%SITEWATCH_BASE_DIR%\classes
set SITEWATCH_LIB_DIR=%SITEWATCH_BASE_DIR%\lib

set M4_CLASSPATH=%SITEWATCH_CLS_DIR%
for %%f in (%SITEWATCH_LIB_DIR%\*.jar) do call :buildClassPath %%f
goto run
:buildClassPath
set M4_CLASSPATH=%M4_CLASSPATH%;%1
goto :EOF

:run

set TOMCATDIR=c:\tomcat-5.5.28
set M4_CLASSPATH=%M4_CLASSPATH%;%TOMCATDIR%\common\lib\servlet-
api.jar

java -cp %M4_CLASSPATH%
com.meta4.sitewatcher.manager.M4ExecuteNotifier
M4GatewayConnectNotifier1
```

Si la prueba se ejecuta con éxito aparecerá en pantalla el valor 0, si no, un 1.

Como se puede ver, hace falta la dependencia del API de Servlet 2.x (archivo servlet.jar o servlet-api.jar).